

Trabajo Práctico Final: Predicción de Precios Hoteleros

Taller de Resolución de Problemas I - Caso Despegar

Alumnas: Narda Rosa, Julieta Kimel (2do Año) | Profesores: Mariano Esmail, Andrés Farall

Universidad Austral - Lic. en Ciencia de Datos

Contents

1	=====	
1.1	Resumen Ejecutivo (Abstract)	2
2	=====	
3	=====	
3.1	1. Directorio de trabajo y librerías	3
4	=====	
5	=====	
5.1	2. Carga de datasets	3
6	=====	
6.0.1	Dataset: Copia de searches_6381	3
6.0.2	Dataset: Amenities Descriptions	3
7	=====	
7.1	3. Exploración de cada dataset	3
8	=====	
8.0.1	3.1 Searches 6381 - Dimensiones y columnas	3
8.0.2	3.2 Amenities Descriptions - Dimensiones y columnas	5
9	=====	
9.1	4. Unión de los dos datasets	6
10	=====	
11	=====	
11.1	5. Análisis Exploratorio de Datos (EDA)	9
12	=====	
12.0.1	5.1 Estructura general del dataset	9
12.0.2	5.2 Valores faltantes (NA)	12
12.0.3	5.3 Variable objetivo: target	14
12.0.4	5.4 Distribución de precios	15
12.0.5	5.5 Estrellas del hotel	17
12.0.6	5.6 Ratings del hotel	18
12.0.7	5.7 Países de origen de las búsquedas	19

12.0.8	5.8 Anticipacion de reserva	20
12.0.9	5.9 Duracion de la estadia	21
12.0.10	5.10 Estacionalidad: busquedas por mes de check-in	22
12.0.11	5.11 Matriz de correlacion entre variables numericas	23
12.0.12	5.12 Posicion en resultados vs. compra	24
13	=====	
13.1	6. Ingenieria de Features	25
14	=====	
14.0.1	6.1 Filtrar solo Rio de Janeiro	25
14.0.2	6.2 Variable objetivo correcta	25
14.0.3	6.3 Features temporales desde date	26
14.0.4	6.4 Normalizacion de precios (precio relativo)	27
14.0.5	6.5 Features geograficas: distancia a puntos de referencia	27
14.0.6	6.6 Features de amenities (dummies por categoria)	29
14.0.7	6.7 Feature de busquedas por hotel (proxy de popularidad)	30
14.0.8	6.8 Manejo de outliers y NAs	30
14.0.9	6.9 Segmentacion geografica supervisada	31
15	=====	
15.1	Metodología	32
16	=====	
16.0.1	Limpieza Final y Prevención de Data Leakage	32
16.0.2	Partición de Datos (Train / Test)	34
17	=====	
17.1	Resultados	35
18	=====	
18.0.1	Línea Base Estocástica: Regresión Lineal Simple	35
18.0.2	Ensambls Arbóreos: Implementación de Random Forest	38
18.0.3	Evaluación del Desempeño sobre Datos Inéditos	39
19	=====	
19.1	Discusión y Conclusiones	42
20	=====	
20.0.1	Análisis Estructural: Importancia de Atributos	42
20.0.2	Conclusión Académica	44

1

1.1 Resumen Ejecutivo (Abstract)

2

Análisis: El presente documento detalla la formulación y evaluación de un modelo predictivo orientado a estimar el precio por persona por noche en establecimientos hoteleros de la ciudad de Río de Janeiro. El objetivo central consiste en identificar los atributos de mayor peso en la conformación del precio, empleando técnicas de Machine Learning para capturar tanto relaciones lineales como interacciones complejas. Para ello, se aplicó un preprocesamiento riguroso orientado a prevenir la filtración de datos futuros (*data leakage*). Posteriormente, se estableció como línea base un modelo de regresión lineal simple y se procedió a entrenar un algoritmo *Random Forest*,

el cual demostró una superioridad notable en la capacidad de generalización. Los resultados obtenidos permiten extraer valiosas conclusiones empíricas sobre la dinámica de precios en el sector turístico de la región.

3

3.1 1. Directorio de trabajo y librerías

4

```
# Fija el directorio al ejecutar chunks de forma interactiva en RStudio
# NOTA: eval=FALSE - este chunk solo se ejecuta manualmente en RStudio.
#       Al usar rmarkdown::render() desde consola no es necesario porque
#       el directorio de trabajo se fija antes de llamar a render().
setwd(dirname(rstudioapi::getActiveDocumentContext()$path))
getwd() # Verificamos que sea correcto
```

```
library(tidyverse)
library(corrplot)
library(lubridate)
if (!requireNamespace("geosphere", quietly = TRUE)) {
  install.packages("geosphere", repos = "https://cloud.r-project.org")
}
library(geosphere)
```

5

5.1 2. Carga de datasets

6

6.0.1 Dataset: Copia de searches_6381

```
searches <- read.csv("../data/raw/searches_6381.csv", stringsAsFactors = FALSE)
```

6.0.2 Dataset: Amenities Descriptions

```
amenities <- read.csv("../data/raw/amenities_descriptions.csv", stringsAsFactors = FALSE)
```

7

7.1 3. Exploracion de cada dataset

8

8.0.1 3.1 Searches 6381 - Dimensiones y columnas

```
cat("Filas: ", nrow(searches), "\n")
```

```
## Filas: 1184083
```

```
cat("Columnas:", ncol(searches), "\n")
```

```
## Columnas: 41
```

```
cat("Columnas del dataset Searches:\n")
```

```
## Columnas del dataset Searches:
```

```
print(colnames(searches))
```

```
## [1] "searchid"           "date"
## [3] "hid"                "position"
## [5] "adults"             "children"
## [7] "infants"            "duration"
## [9] "anticipation"       "AR"
## [11] "BR"                 "CL"
## [13] "CO"                 "MX"
## [15] "PE"                 "UY"
## [17] "OTHER"              "price_by_night"
## [19] "numberOfRooms"      "starRating"
## [21] "amen_ohe_extraflag" "avgRating"
## [23] "avgRatingCleaning"  "avgRatingInternetAccessAndQuality"
## [25] "avgRatingLocation"  "avgQualityprice"
## [27] "avgServicepersonal" "avgService"
## [29] "month_ci"           "month_ci_sine"
## [31] "month_ci_cosine"    "geo_trunc_lat"
## [33] "geo_trunc_lon"      "price_by_night_adult"
## [35] "price_by_night_person" "min_query_price"
## [37] "target"             "detail"
## [39] "latitude"           "longitude"
## [41] "geo_id"
```

```
head(searches)
```

```
##           searchid      date      hid position adults
## 1 3b72501c-0de5-4fda-bedd-19053cbb4fb7 2024-08-20 983079      7      2
## 2 3b72501c-0de5-4fda-bedd-19053cbb4fb7 2024-08-20 487165      2      2
## 3 3b72501c-0de5-4fda-bedd-19053cbb4fb7 2024-08-20 231438      5      2
## 4 3b72501c-0de5-4fda-bedd-19053cbb4fb7 2024-08-20 4479125      3      2
## 5 3b72501c-0de5-4fda-bedd-19053cbb4fb7 2024-08-20 486646      8      2
## 6 3b72501c-0de5-4fda-bedd-19053cbb4fb7 2024-08-20 868613     10      2
##  children infants duration anticipation AR BR CL CO MX PE UY OTHER
## 1      0      0      1      4 0 1 0 0 0 0 0 0
## 2      0      0      1      4 0 1 0 0 0 0 0 0
## 3      0      0      1      4 0 1 0 0 0 0 0 0
## 4      0      0      1      4 0 1 0 0 0 0 0 0
## 5      0      0      1      4 0 1 0 0 0 0 0 0
## 6      0      0      1      4 0 1 0 0 0 0 0 0
## price_by_night numberOfRooms starRating amen_ohe_extraflag avgRating
## 1      116.03843      303      4      0 89.73148
## 2      87.58315      195      4      0 71.98327
## 3      158.72136      115      4      0 87.17949
## 4      156.87361      378      4      0 89.71429
## 5      127.30968      59      4      0 75.92593
## 6      226.16408      131      5      0 82.88889
## avgRatingCleaning avgRatingInternetAccessAndQuality avgRatingLocation
```

```

## 1      89.50000      88.08333      90.41667
## 2      65.48387      72.25806      84.83871
## 3      87.69231      86.15385      92.82051
## 4      90.28571      90.28571      90.85714
## 5      70.83333      81.66667      90.83333
## 6      83.00000      84.00000      85.00000
## avgQualityprice avgServicepersonal avgService month_ci month_ci_sine
## 1      86.66667      92.08333      90.50000      8      -0.86602
## 2      70.75269      73.33333      67.20430      8      -0.86602
## 3      85.64103      94.35897      83.07692      8      -0.86602
## 4      84.00000      91.42857      92.00000      8      -0.86602
## 5      67.50000      78.33333      69.16667      8      -0.86602
## 6      73.00000      88.00000      85.00000      8      -0.86602
## month_ci_cosine geo_trunc_lat geo_trunc_lon price_by_night_adult
## 1      -0.5      -23.00      -43.35      58.01922
## 2      -0.5      -22.97      -43.19      43.79157
## 3      -0.5      -22.98      -43.18      79.36068
## 4      -0.5      -23.01      -43.32      78.43681
## 5      -0.5      -22.96      -43.17      63.65484
## 6      -0.5      -22.94      -43.18      113.08204
## price_by_night_person min_query_price target detail latitude longitude
## 1      58.01922      30.39542      0      0 -23.00969 -43.35319
## 2      43.79157      30.39542      0      0 -22.97600 -43.19308
## 3      79.36068      30.39542      0      0 -22.98507 -43.18982
## 4      78.43681      30.39542      0      0 -23.01056 -43.32164
## 5      63.65484      30.39542      0      0 -22.96463 -43.17841
## 6      113.08204      30.39542      0      0 -22.94298 -43.18142
## geo_id
## 1      6381
## 2      6381
## 3      6381
## 4      6381
## 5      6381
## 6      6381

```

8.0.2 3.2 Amenities Descriptions - Dimensiones y columnas

```
cat("Filas:  ", nrow(amenities), "\n")
```

```
## Filas:      159
```

```
cat("Columnas:", ncol(amenities), "\n")
```

```
## Columnas: 8
```

```
cat("Columnas del dataset Amenities:\n")
```

```
## Columnas del dataset Amenities:
```

```
print(colnames(amenities))
```

```

## [1] "id"                                "descriptions.pt"
## [3] "descriptions.en"                  "descriptions.es"
## [5] "amenity_category.id"              "amenity_category.descriptions.pt"
## [7] "amenity_category.descriptions.en" "amenity_category.descriptions.es"

```

```
head(amenities)
```

```
##          id                                descriptions.pt
## 1 PARKADCO                      Estacionamento com custo adicional
## 2 VENDMACH                      Máquina de venda automática
## 3 BIKECO                        Bicicletas disponíveis com custo adicional
## 4 SERGUAR                      Serviço de babá
## 5 SNA                          Lanchonete
## 6 TRABESE Exclusivo para trabalhadores de atividades essenciais
##          descriptions.en
## 1          Parking with additional cost
## 2          Vending machine
## 3 Bikes available with additional cost
## 4          Babysitting services
## 5          Snack bar
## 6          Essential Workers Only
##          descriptions.es amenity_category.id
## 1          Estacionamento com custo adicional          ESTA
## 2          Máquina expendedora          COMU
## 3          Bicicletas disponibles con Costo Adicional          ESTA
## 4          Servicio de niñera          NINO
## 5          Snack bar          ALIM
## 6 Exclusivo para trabajadores de actividades esenciales          SST
## amenity_category.descriptions.pt amenity_category.descriptions.en
## 1          Estacionamento e transporte          Parking and Mobility
## 2          Serviços em áreas comuns          Services in common areas
## 3          Estacionamento e transporte          Parking and Mobility
## 4          Para crianças          Facilities for kids
## 5          Refeições          Food & Drinks
## 6          Higiene e segurança          Sanitary measures
## amenity_category.descriptions.es
## 1          Estacionamiento y movilidad
## 2          Servicios en áreas comunes
## 3          Estacionamiento y movilidad
## 4          Para niños
## 5          Alimentación
## 6          Medidas de higiene y seguridad
```

9

9.1 4. Union de los dos datasets

10

Nota: `amen_ohe_extraflag` en `searches` es una variable binaria (0/1) y no es una clave foranea directa hacia `id` de `amenities` (que contiene codigos como “PARKADCO”).

Por eso se unen convirtiendo ambas columnas al mismo tipo (character) para evitar el error de tipos.

Las filas de `searches` con valor 0 o 1 no coincidirán con los codigos de `amenities`, por lo que el `left join` preservara todas las filas de `searches` agregando NA en las columnas de `amenities`.

```

# Convertimos amen_ohe_extraflag a character para que los tipos sean compatibles
searches_chr <- searches %>%
  mutate(amen_ohe_extraflag = as.character(amen_ohe_extraflag))

amenities_join <- amenities %>%
  rename(amen_ohe_extraflag = id) # id ya es character

# Left join: conserva TODAS las filas de searches
dataset_unido <- searches_chr %>%
  left_join(amenities_join, by = "amen_ohe_extraflag")

cat("Filas del dataset unido: ", nrow(dataset_unido), "\n")

## Filas del dataset unido: 1184083

cat("Columnas del dataset unido:", ncol(dataset_unido), "\n")

## Columnas del dataset unido: 48

# Cuántas filas tuvieron match con amenities?
matches <- sum(!is.na(dataset_unido$descriptions.es))
cat("Filas con amenity asociada:", matches, "\n")

## Filas con amenity asociada: 0

cat("Filas sin match (NA): ", nrow(dataset_unido) - matches, "\n")

## Filas sin match (NA): 1184083

cat("Columnas del dataset unido:\n")

## Columnas del dataset unido:
print(colnames(dataset_unido))

## [1] "searchid" "date"
## [3] "hid" "position"
## [5] "adults" "children"
## [7] "infants" "duration"
## [9] "anticipation" "AR"
## [11] "BR" "CL"
## [13] "CO" "MX"
## [15] "PE" "UY"
## [17] "OTHER" "price_by_night"
## [19] "numberOfRooms" "starRating"
## [21] "amen_ohe_extraflag" "avgRating"
## [23] "avgRatingCleaning" "avgRatingInternetAccessAndQuality"
## [25] "avgRatingLocation" "avgQualityprice"
## [27] "avgServicepersonal" "avgService"
## [29] "month_ci" "month_ci_sine"
## [31] "month_ci_cosine" "geo_trunc_lat"
## [33] "geo_trunc_lon" "price_by_night_adult"
## [35] "price_by_night_person" "min_query_price"
## [37] "target" "detail"
## [39] "latitude" "longitude"
## [41] "geo_id" "descriptions.pt"
## [43] "descriptions.en" "descriptions.es"
## [45] "amenity_category.id" "amenity_category.descriptions.pt"

```

```
## [47] "amenity_category.descriptions.en" "amenity_category.descriptions.es"
```

```
head(dataset_unido)
```

```
##           searchid      date      hid position adults
## 1 3b72501c-0de5-4fda-bedd-19053cbb4fb7 2024-08-20 983079         7      2
## 2 3b72501c-0de5-4fda-bedd-19053cbb4fb7 2024-08-20 487165         2      2
## 3 3b72501c-0de5-4fda-bedd-19053cbb4fb7 2024-08-20 231438         5      2
## 4 3b72501c-0de5-4fda-bedd-19053cbb4fb7 2024-08-20 4479125        3      2
## 5 3b72501c-0de5-4fda-bedd-19053cbb4fb7 2024-08-20 486646         8      2
## 6 3b72501c-0de5-4fda-bedd-19053cbb4fb7 2024-08-20 868613        10      2
##   children infants duration anticipation AR BR CL CO MX PE UY OTHER
## 1         0         0         1           4 0 1 0 0 0 0 0 0
## 2         0         0         1           4 0 1 0 0 0 0 0 0
## 3         0         0         1           4 0 1 0 0 0 0 0 0
## 4         0         0         1           4 0 1 0 0 0 0 0 0
## 5         0         0         1           4 0 1 0 0 0 0 0 0
## 6         0         0         1           4 0 1 0 0 0 0 0 0
##   price_by_night numberOfRooms starRating amen_ohe_extraflag avgRating
## 1      116.03843           303           4              0 89.73148
## 2       87.58315           195           4              0 71.98327
## 3      158.72136           115           4              0 87.17949
## 4      156.87361           378           4              0 89.71429
## 5      127.30968            59           4              0 75.92593
## 6      226.16408           131           5              0 82.88889
##   avgRatingCleaning avgRatingInternetAccessAndQuality avgRatingLocation
## 1      89.50000              88.08333          90.41667
## 2      65.48387              72.25806          84.83871
## 3      87.69231              86.15385          92.82051
## 4      90.28571              90.28571          90.85714
## 5      70.83333              81.66667          90.83333
## 6      83.00000              84.00000          85.00000
##   avgQualityprice avgServicepersonal avgService month_ci month_ci_sine
## 1      86.66667           92.08333  90.50000         8    -0.86602
## 2      70.75269           73.33333  67.20430         8    -0.86602
## 3      85.64103           94.35897  83.07692         8    -0.86602
## 4      84.00000           91.42857  92.00000         8    -0.86602
## 5      67.50000           78.33333  69.16667         8    -0.86602
## 6      73.00000           88.00000  85.00000         8    -0.86602
##   month_ci_cosine geo_trunc_lat geo_trunc_lon price_by_night_adult
## 1          -0.5      -23.00      -43.35      58.01922
## 2          -0.5      -22.97      -43.19      43.79157
## 3          -0.5      -22.98      -43.18      79.36068
## 4          -0.5      -23.01      -43.32      78.43681
## 5          -0.5      -22.96      -43.17      63.65484
## 6          -0.5      -22.94      -43.18     113.08204
##   price_by_night_person min_query_price target detail latitude longitude
## 1      58.01922      30.39542      0      0 -23.00969 -43.35319
## 2      43.79157      30.39542      0      0 -22.97600 -43.19308
## 3      79.36068      30.39542      0      0 -22.98507 -43.18982
## 4      78.43681      30.39542      0      0 -23.01056 -43.32164
## 5      63.65484      30.39542      0      0 -22.96463 -43.17841
## 6     113.08204      30.39542      0      0 -22.94298 -43.18142
##   geo_id descriptions.pt descriptions.en descriptions.es amenity_category.id
## 1    6381          <NA>          <NA>          <NA>          <NA>
```

```
## 2 6381 <NA> <NA> <NA> <NA>
## 3 6381 <NA> <NA> <NA> <NA>
## 4 6381 <NA> <NA> <NA> <NA>
## 5 6381 <NA> <NA> <NA> <NA>
## 6 6381 <NA> <NA> <NA> <NA>
## amenity_category.descriptions.pt amenity_category.descriptions.en
## 1 <NA> <NA>
## 2 <NA> <NA>
## 3 <NA> <NA>
## 4 <NA> <NA>
## 5 <NA> <NA>
## 6 <NA> <NA>
## amenity_category.descriptions.es
## 1 <NA>
## 2 <NA>
## 3 <NA>
## 4 <NA>
## 5 <NA>
## 6 <NA>
```

11

11.1 5. Analisis Exploratorio de Datos (EDA)

12

12.0.1 5.1 Estructura general del dataset

```
glimpse(dataset_unido)
```

```
## Rows: 1,184,083
## Columns: 48
## $ searchid <chr> "3b72501c-0de5-4fda-bedd-19053cbb4fb~
## $ date <chr> "2024-08-20", "2024-08-20", "2024-08~
## $ hid <int> 983079, 487165, 231438, 4479125, 486~
## $ position <int> 7, 2, 5, 3, 8, 10, 6, 9, 1, 4, 4, 3, ~
## $ adults <int> 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, ~
## $ children <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
## $ infants <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
## $ duration <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
## $ anticipation <int> 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 32, 32~
## $ AR <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
## $ BR <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
## $ CL <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
## $ CO <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
## $ MX <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
## $ PE <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
## $ UY <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
## $ OTHER <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
## $ price_by_night <dbl> 116.03843, 87.58315, 158.72136, 156.~
## $ numberOfRooms <int> 303, 195, 115, 378, 59, 131, 249, 10~
## $ starRating <dbl> 4, 4, 4, 4, 4, 5, 4, 4, 4, 5, 4, 5, ~
```

```
## $ amen_ohe_extraflag      <chr> "0", "0", "0", "0", "0", "0", "0", "~
## $ avgRating               <dbl> 89.73148, 71.98327, 87.17949, 89.714~
## $ avgRatingCleaning       <dbl> 89.50000, 65.48387, 87.69231, 90.285~
## $ avgRatingInternetAccessAndQuality <dbl> 88.08333, 72.25806, 86.15385, 90.285~
## $ avgRatingLocation       <dbl> 90.41667, 84.83871, 92.82051, 90.857~
## $ avgQualityprice         <dbl> 86.66667, 70.75269, 85.64103, 84.000~
## $ avgServicepersonal      <dbl> 92.08333, 73.33333, 94.35897, 91.428~
## $ avgService              <dbl> 90.50000, 67.20430, 83.07692, 92.000~
## $ month_ci                <int> 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 10, 10~
## $ month_ci_sine           <dbl> -0.86602, -0.86602, -0.86602, -0.866~
## $ month_ci_cosine         <dbl> -0.50000, -0.50000, -0.50000, -0.500~
## $ geo_trunc_lat           <dbl> -23.00, -22.97, -22.98, -23.01, -22.~
## $ geo_trunc_lon           <dbl> -43.35, -43.19, -43.18, -43.32, -43.~
## $ price_by_night_adult     <dbl> 58.01922, 43.79157, 79.36068, 78.436~
## $ price_by_night_person    <dbl> 58.01922, 43.79157, 79.36068, 78.436~
## $ min_query_price          <dbl> 30.39542, 30.39542, 30.39542, 30.395~
## $ target                  <int> 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, ~
## $ detail                  <int> 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, ~
## $ latitude                <dbl> -23.00969, -22.97600, -22.98507, -23~
## $ longitude               <dbl> -43.35319, -43.19308, -43.18982, -43~
## $ geo_id                  <int> 6381, 6381, 6381, 6381, 6381, 6381, ~
## $ descriptions.pt          <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ descriptions.en          <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ descriptions.es          <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ amenity_category.id      <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ amenity_category.descriptions.pt <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ amenity_category.descriptions.en <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ amenity_category.descriptions.es <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
```

```
summary(dataset_unido)
```

```
##      searchid      date      hid      position
## Length:1184083 Length:1184083 Min.   : 231000 Min.   : 1.000
## Class :character Class :character 1st Qu.: 231814 1st Qu.: 4.000
## Mode  :character Mode  :character Median : 661634 Median : 8.000
##                                     Mean  : 912384 Mean   : 9.215
##                                     3rd Qu.: 902124 3rd Qu.:14.000
##                                     Max.   :6786600 Max.    :21.000
##
##      adults      children      infants      duration
## Min.   : 1.00   Min.   :0.00000 Min.   :0.00000 Min.   : 1.000
## 1st Qu.: 2.00   1st Qu.:0.00000 1st Qu.:0.00000 1st Qu.: 1.000
## Median : 2.00   Median :0.00000 Median :0.00000 Median : 3.000
## Mean   : 2.03   Mean   :0.1785 Mean   :0.02201 Mean   : 3.025
## 3rd Qu.: 2.00   3rd Qu.:0.00000 3rd Qu.:0.00000 3rd Qu.: 4.000
## Max.   :16.00   Max.   :6.0000 Max.   :3.00000 Max.   :30.000
##
##      anticipation      AR      BR      CL
## Min.   : -1.00   Min.   :0.00000 Min.   :0.00000 Min.   :0.00000
## 1st Qu.:  4.00   1st Qu.:0.00000 1st Qu.:0.00000 1st Qu.:0.00000
## Median : 18.00   Median :0.00000 Median :1.00000 Median :0.00000
## Mean   : 37.34   Mean   :0.1821 Mean   :0.7191 Mean   :0.06292
## 3rd Qu.: 51.00   3rd Qu.:0.00000 3rd Qu.:1.00000 3rd Qu.:0.00000
## Max.   :545.00   Max.   :1.00000 Max.   :1.00000 Max.   :1.00000
##
```

```

##          CO                MX                PE                UY
## Min.    :0.00000    Min.    :0.000000    Min.    :0.00000    Min.    :0.000000
## 1st Qu.:0.00000    1st Qu.:0.000000    1st Qu.:0.00000    1st Qu.:0.000000
## Median :0.00000    Median :0.000000    Median :0.00000    Median :0.000000
## Mean    :0.01107    Mean    :0.009415    Mean    :0.01046    Mean    :0.002751
## 3rd Qu.:0.00000    3rd Qu.:0.000000    3rd Qu.:0.00000    3rd Qu.:0.000000
## Max.    :1.00000    Max.    :1.000000    Max.    :1.00000    Max.    :1.000000
##
##          OTHER          price_by_night    numberOfRooms    starRating
## Min.    :0.000000    Min.    :    5.996    Min.    :    0.0    Min.    :0.000
## 1st Qu.:0.000000    1st Qu.:   81.049    1st Qu.:  105.0    1st Qu.:4.000
## Median :0.000000    Median :  114.425    Median :  204.0    Median :4.000
## Mean    :0.002214    Mean    :  170.311    Mean    :  226.2    Mean    :3.844
## 3rd Qu.:0.000000    3rd Qu.:  174.264    3rd Qu.:  291.0    3rd Qu.:4.000
## Max.    :1.000000    Max.    :13841.514    Max.    :1406.0    Max.    :5.000
##
## amen_ohe_extraflag    avgRating    avgRatingCleaning
## Length:1184083    Min.    :49.71    Min.    : 44.29
## Class :character    1st Qu.:80.81    1st Qu.: 79.87
## Mode  :character    Median :84.69    Median : 84.11
##                               Mean    :83.71    Mean    : 83.20
##                               3rd Qu.:88.22    3rd Qu.: 87.70
##                               Max.    :96.67    Max.    :100.00
##                               NA's    :63453    NA's    :63453
## avgRatingInternetAccessAndQuality    avgRatingLocation    avgQualityprice
## Min.    : 28.57                                Min.    : 43.00    Min.    : 51.43
## 1st Qu.: 74.17                                1st Qu.: 85.35    1st Qu.: 78.00
## Median : 80.25                                Median : 89.71    Median : 82.61
## Mean    : 78.99                                Mean    : 88.39    Mean    : 81.14
## 3rd Qu.: 85.13                                3rd Qu.: 93.48    3rd Qu.: 85.25
## Max.    :100.00                                Max.    :100.00    Max.    :100.00
## NA's    :63453                                NA's    :63453    NA's    :63453
## avgServicepersonal    avgService    month_ci    month_ci_sine
## Min.    : 50.00    Min.    : 47.00    Min.    : 1.000    Min.    : -1.00000
## 1st Qu.: 84.71    1st Qu.: 79.46    1st Qu.: 4.000    1st Qu.: -0.50000
## Median : 88.52    Median : 83.81    Median : 6.000    Median : 0.00000
## Mean    : 87.11    Mean    : 83.37    Mean    : 6.144    Mean    : 0.07401
## 3rd Qu.: 91.94    3rd Qu.: 88.55    3rd Qu.: 8.000    3rd Qu.: 0.86602
## Max.    :100.00    Max.    :100.00    Max.    :12.000    Max.    : 1.00000
## NA's    :63453    NA's    :63453
## month_ci_cosine    geo_trunc_lat    geo_trunc_lon    price_by_night_adult
## Min.    : -1.0000    Min.    : -23.07    Min.    : -44.51    Min.    :    2.335
## 1st Qu.: -0.8660    1st Qu.: -23.00    1st Qu.: -43.31    1st Qu.:   43.201
## Median : -0.5000    Median : -22.97    Median : -43.18    Median :   61.995
## Mean    : -0.1894    Mean    : -22.97    Mean    : -43.23    Mean    :   90.422
## 3rd Qu.: 0.5000    3rd Qu.: -22.96    3rd Qu.: -43.18    3rd Qu.:   93.306
## Max.    : 1.0000    Max.    : -22.30    Max.    : -42.30    Max.    :13841.514
##
## price_by_night_person    min_query_price    target    detail
## Min.    :    2.335    Min.    : 2.335    Min.    :0.00000    Min.    :0.00000
## 1st Qu.: 40.261    1st Qu.: 19.471    1st Qu.:0.00000    1st Qu.:0.00000
## Median : 57.824    Median : 27.981    Median :0.00000    Median :0.00000
## Mean    : 84.168    Mean    : 31.735    Mean    :0.04322    Mean    :0.05732
## 3rd Qu.: 87.461    3rd Qu.: 38.502    3rd Qu.:0.00000    3rd Qu.:0.00000

```

```
## Max. :13841.514      Max. :350.856      Max. :1.00000      Max. :1.00000
##
## latitude longitude geo_id descriptions.pt
## Min. :-23.07 Min. :-44.52 Min. :6381 Length:1184083
## 1st Qu.:-23.00 1st Qu.:-43.32 1st Qu.:6381 Class :character
## Median :-22.97 Median :-43.19 Median :6381 Mode :character
## Mean :-22.97 Mean :-43.24 Mean :6381
## 3rd Qu.:-22.96 3rd Qu.:-43.18 3rd Qu.:6381
## Max. :-22.30 Max. :-42.31 Max. :6381
## NA's :1154 NA's :1154
## descriptions.en descriptions.es amenity_category.id
## Length:1184083 Length:1184083 Length:1184083
## Class :character Class :character Class :character
## Mode :character Mode :character Mode :character
##
##
##
## amenity_category.descriptions.pt amenity_category.descriptions.en
## Length:1184083 Length:1184083
## Class :character Class :character
## Mode :character Mode :character
##
##
##
## amenity_category.descriptions.es
## Length:1184083
## Class :character
## Mode :character
##
##
##
##
```

12.0.2 5.2 Valores faltantes (NA)

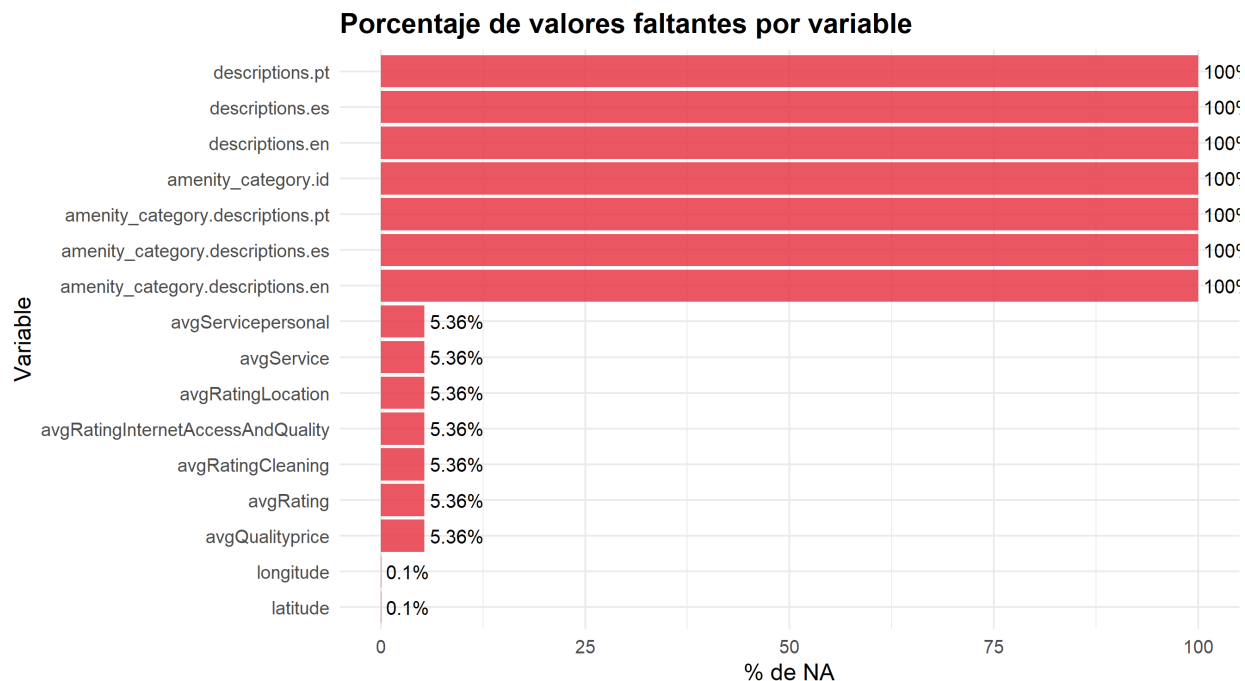
```
# Conteo y porcentaje de NA por columna
na_df <- dataset_unido %>%
  summarise(across(everything(), ~ sum(is.na(.)))) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "na_count") %>%
  mutate(pct_na = round(na_count / nrow(dataset_unido) * 100, 2)) %>%
  filter(na_count > 0) %>%
  arrange(desc(pct_na))

print(na_df)
```

```
## # A tibble: 16 x 3
##   variable na_count pct_na
##   <chr>      <int> <dbl>
## 1 descriptions.pt 1184083 100
## 2 descriptions.en 1184083 100
## 3 descriptions.es 1184083 100
```

```
## 4 amenity_category.id          1184083 100
## 5 amenity_category.descriptions.pt 1184083 100
## 6 amenity_category.descriptions.en 1184083 100
## 7 amenity_category.descriptions.es 1184083 100
## 8 avgRating                    63453  5.36
## 9 avgRatingCleaning             63453  5.36
## 10 avgRatingInternetAccessAndQuality 63453  5.36
## 11 avgRatingLocation            63453  5.36
## 12 avgQualityprice              63453  5.36
## 13 avgServicepersonal           63453  5.36
## 14 avgService                   63453  5.36
## 15 latitude                     1154   0.1
## 16 longitude                     1154   0.1
```

```
if (nrow(na_df) > 0) {
  ggplot(na_df, aes(x = reorder(variable, pct_na), y = pct_na)) +
    geom_col(fill = "#E63946", alpha = 0.85) +
    geom_text(aes(label = paste0(pct_na, "%")), hjust = -0.1, size = 3.5) +
    coord_flip() +
    labs(
      title = "Porcentaje de valores faltantes por variable",
      x = "Variable", y = "% de NA"
    ) +
    theme_minimal(base_size = 12) +
    theme(plot.title = element_text(face = "bold"))
} else {
  cat("No hay valores faltantes en el dataset.\n")
}
```



Análisis: El gráfico muestra cuáles variables concentran la mayor proporción de datos faltantes. Las columnas provenientes del join con amenities (como `descriptions.es`) presentan un alto porcentaje de NA ya que la mayoría de las búsquedas no tuvieron un amenity asociado. Las

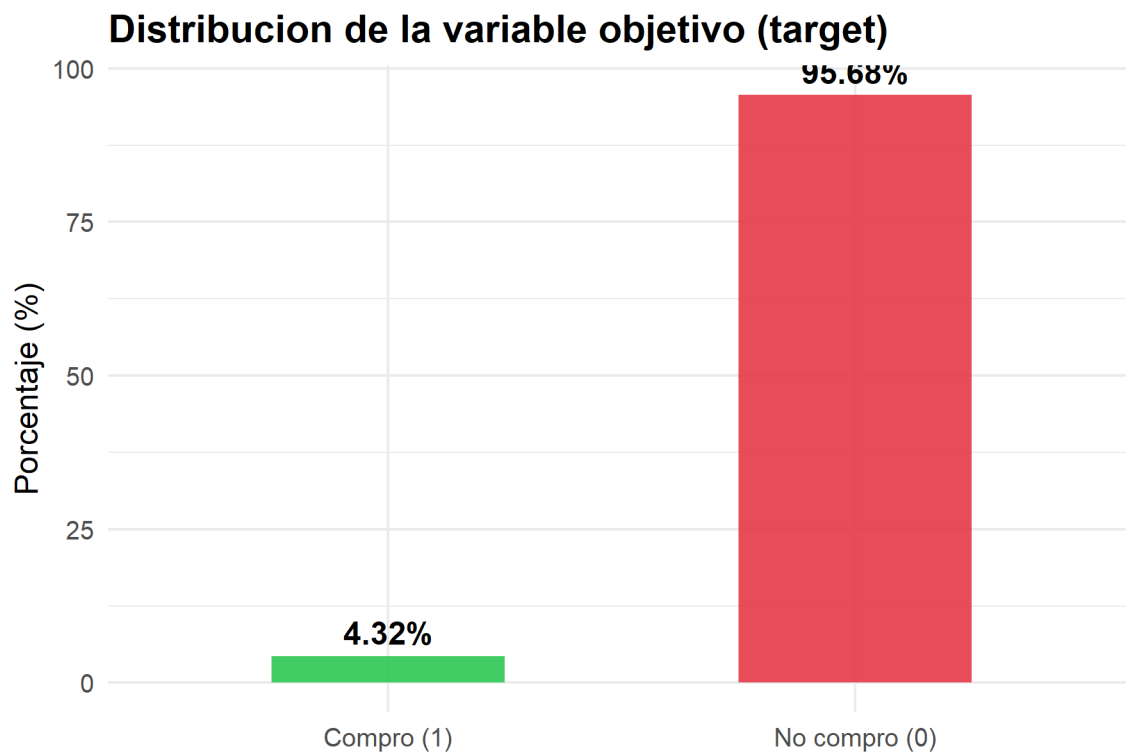
variables con mas del 50% de NA deben evaluarse para decidir si se imputan o se descartan antes del modelado.

12.0.3 5.3 Variable objetivo: target

```
target_resumen <- dataset_unido %>%
  count(target) %>%
  mutate(
    etiqueta = ifelse(target == 1, "Compro (1)", "No compro (0)"),
    porcentaje = round(n / sum(n) * 100, 2)
  )
print(target_resumen)
```

```
##   target      n      etiqueta porcentaje
## 1      0 1132905 No compro (0)      95.68
## 2      1   51178  Compro (1)       4.32
```

```
ggplot(target_resumen, aes(x = etiqueta, y = porcentaje, fill = etiqueta)) +
  geom_col(width = 0.5, alpha = 0.9) +
  geom_text(aes(label = paste0(porcentaje, "%")), vjust = -0.5, fontface = "bold") +
  scale_fill_manual(values = c("Compro (1)" = "#2DC653", "No compro (0)" = "#E63946")) +
  labs(
    title = "Distribucion de la variable objetivo (target)",
    x = NULL, y = "Porcentaje (%)"
  ) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "none", plot.title = element_text(face = "bold"))
```



Análisis: El grafico evidencia un fuerte desbalance de clases: la gran mayoria de las busquedas

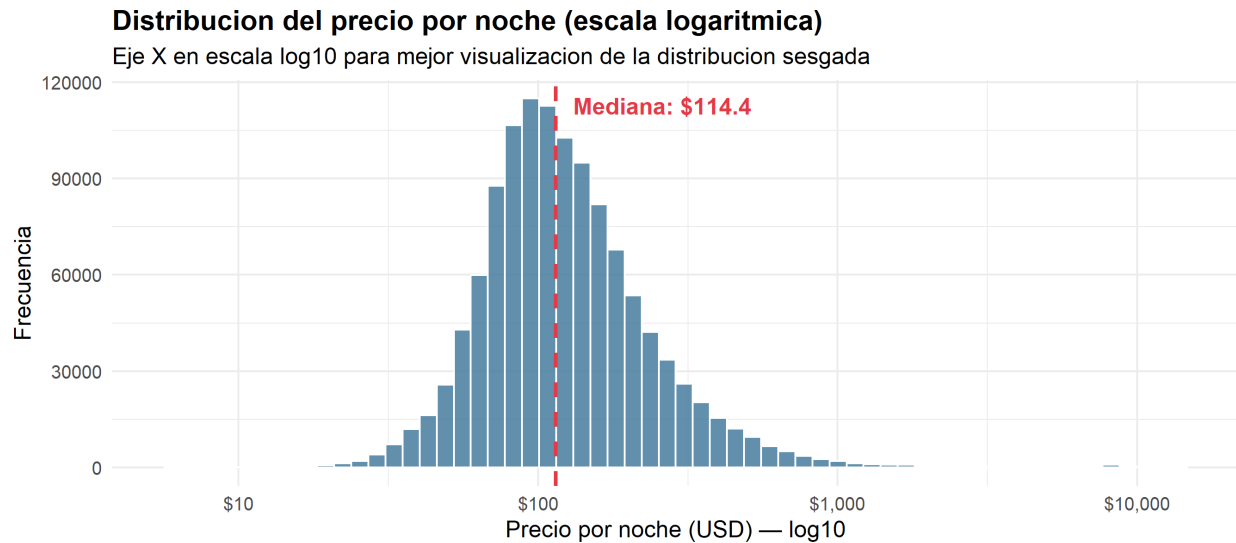
no terminaron en compra. Esta asimetría es típica en datasets de e-commerce y debe tenerse en cuenta al entrenar modelos, ya que un clasificador naive que siempre predice “No compro” obtendría alta accuracy sin ser útil.

12.0.4 5.4 Distribucion de precios

```
dataset_unido %>%
  select(price_by_night, price_by_night_adult, price_by_night_person, min_query_price) %>%
  summary()
```

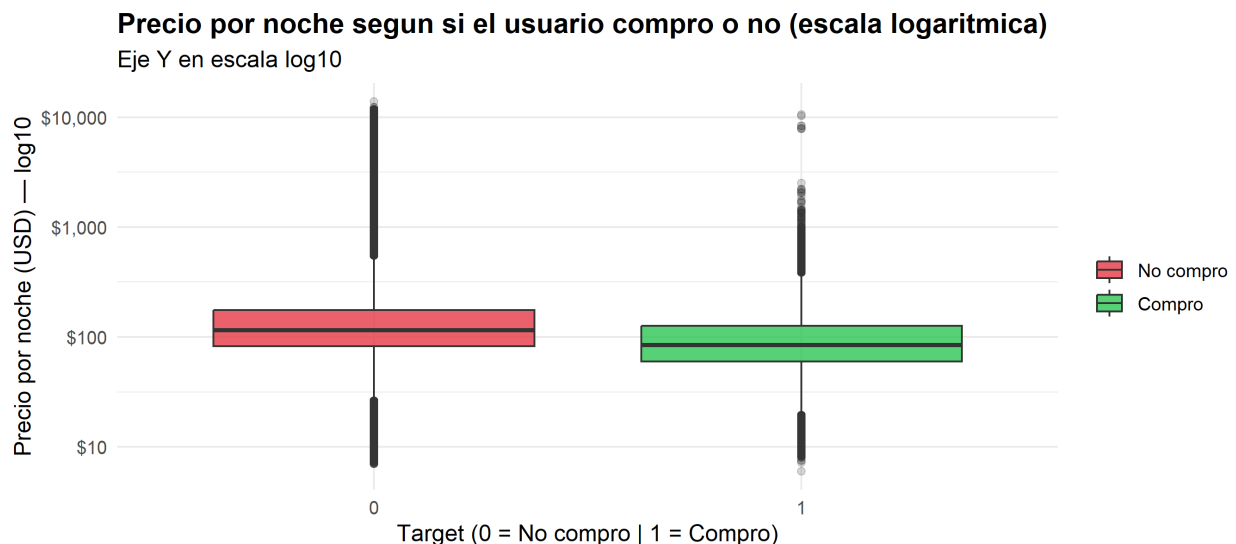
```
## price_by_night      price_by_night_adult price_by_night_person
## Min.   :    5.996      Min.   :    2.335      Min.   :    2.335
## 1st Qu.:   81.049      1st Qu.:   43.201      1st Qu.:   40.261
## Median :  114.425      Median :   61.995      Median :   57.824
## Mean   :  170.311      Mean   :   90.422      Mean   :   84.168
## 3rd Qu.:  174.264      3rd Qu.:   93.306      3rd Qu.:   87.461
## Max.   :13841.514      Max.   :13841.514      Max.   :13841.514
## min_query_price
## Min.   :    2.335
## 1st Qu.:   19.471
## Median :   27.981
## Mean   :   31.735
## 3rd Qu.:   38.502
## Max.   :   350.856
```

```
ggplot(dataset_unido %>% filter(price_by_night > 0), aes(x = price_by_night)) +
  geom_histogram(bins = 60, fill = "#457B9D", color = "white", alpha = 0.85) +
  geom_vline(aes(xintercept = median(price_by_night, na.rm = TRUE)),
    color = "#E63946", linetype = "dashed", linewidth = 1) +
  annotate("text", x = median(dataset_unido$price_by_night, na.rm = TRUE),
    y = Inf, label = paste0("Mediana: $", round(median(dataset_unido$price_by_night, na.rm = TRUE), 2)),
    vjust = 2, hjust = -0.1, color = "#E63946", fontface = "bold") +
  scale_x_log10(labels = scales::dollar_format()) +
  labs(
    title = "Distribucion del precio por noche (escala logaritmica)",
    subtitle = "Eje X en escala log10 para mejor visualizacion de la distribucion sesgada",
    x = "Precio por noche (USD) - log10", y = "Frecuencia"
  ) +
  theme_minimal(base_size = 12) +
  theme(plot.title = element_text(face = "bold"))
```



Análisis: La distribución de precios presenta un fuerte sesgo positivo: la mayoría de los hoteles se concentran en un rango de precio moderado, con algunos outliers de precio muy alto. La escala logaritmica permite visualizar mejor esta distribución. La línea punteada indica la mediana, que resulta mas representativa que la media ante la presencia de valores extremos.

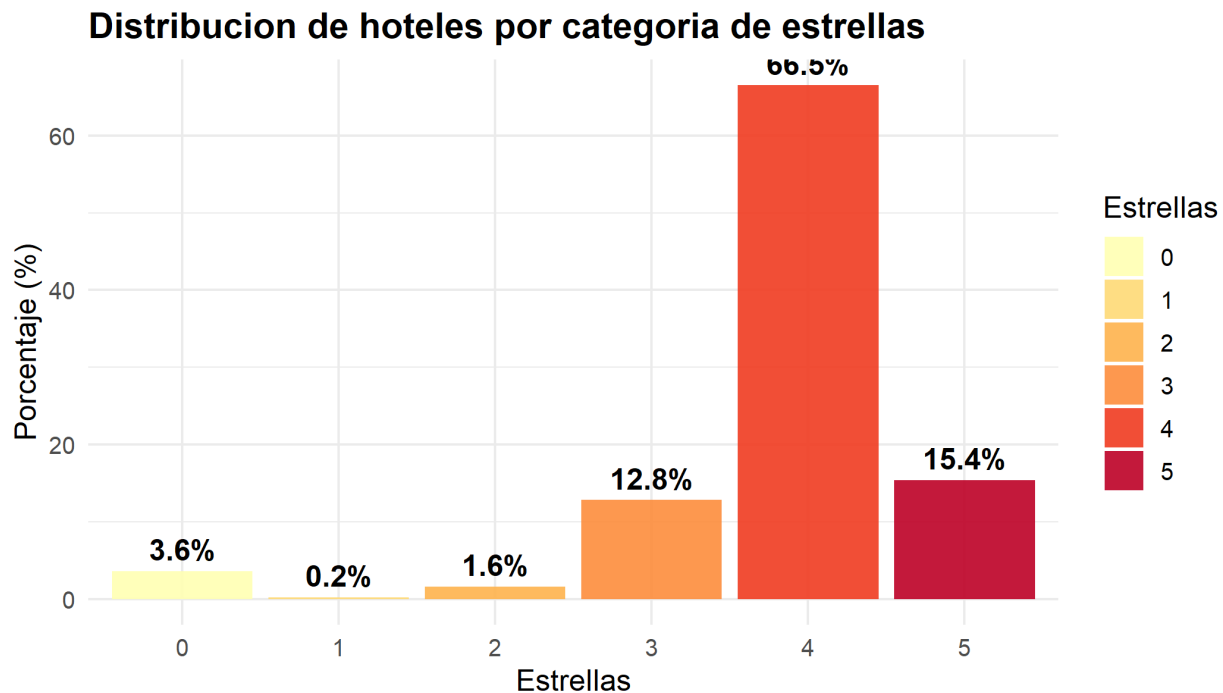
```
ggplot(dataset_unido %>% filter(price_by_night > 0),
  aes(x = factor(target), y = price_by_night, fill = factor(target))) +
  geom_boxplot(alpha = 0.8, outlier.alpha = 0.2) +
  scale_fill_manual(values = c("0" = "#E63946", "1" = "#2DC653"),
    labels = c("No compro", "Compro")) +
  scale_y_log10(labels = scales::dollar_format()) +
  labs(
    title = "Precio por noche segun si el usuario compro o no (escala logaritmica)",
    subtitle = "Eje Y en escala log10",
    x = "Target (0 = No compro | 1 = Compro)", y = "Precio por noche (USD) - log10", fill = NULL
  ) +
  theme_minimal(base_size = 12) +
  theme(plot.title = element_text(face = "bold"))
```



Análisis: Los boxplots muestran como se distribuye el precio segun el resultado de la busqueda. Si la mediana de los hoteles comprados difiere de los no comprados, el precio es una variable con poder discriminativo. En general, los usuarios tienden a comprar hoteles dentro de un rango de precio específico, lo que sugiere que el precio es relevante para el modelo.

12.0.5 5.5 Estrellas del hotel

```
dataset_unido %>%
  count(starRating) %>%
  mutate(pct = round(n / sum(n) * 100, 1)) %>%
  ggplot(aes(x = factor(starRating), y = pct, fill = factor(starRating))) +
  geom_col(alpha = 0.9) +
  geom_text(aes(label = paste0(pct, "%"), vjust = -0.5, fontface = "bold")) +
  scale_fill_brewer(palette = "YlOrRd") +
  labs(
    title = "Distribucion de hoteles por categoria de estrellas",
    x = "Estrellas", y = "Porcentaje (%)", fill = "Estrellas"
  ) +
  theme_minimal(base_size = 12) +
  theme(plot.title = element_text(face = "bold"))
```



Análisis: El grafico muestra que la mayor parte de las busquedas se concentra en hoteles de 3 y 4 estrellas, lo cual refleja la preferencia tipica del usuario de Despegar por opciones de categoria media-alta. La presencia de hoteles sin calificacion (NA o 0) puede indicar registros incompletos que requieren tratamiento.

12.0.6 5.6 Ratings del hotel

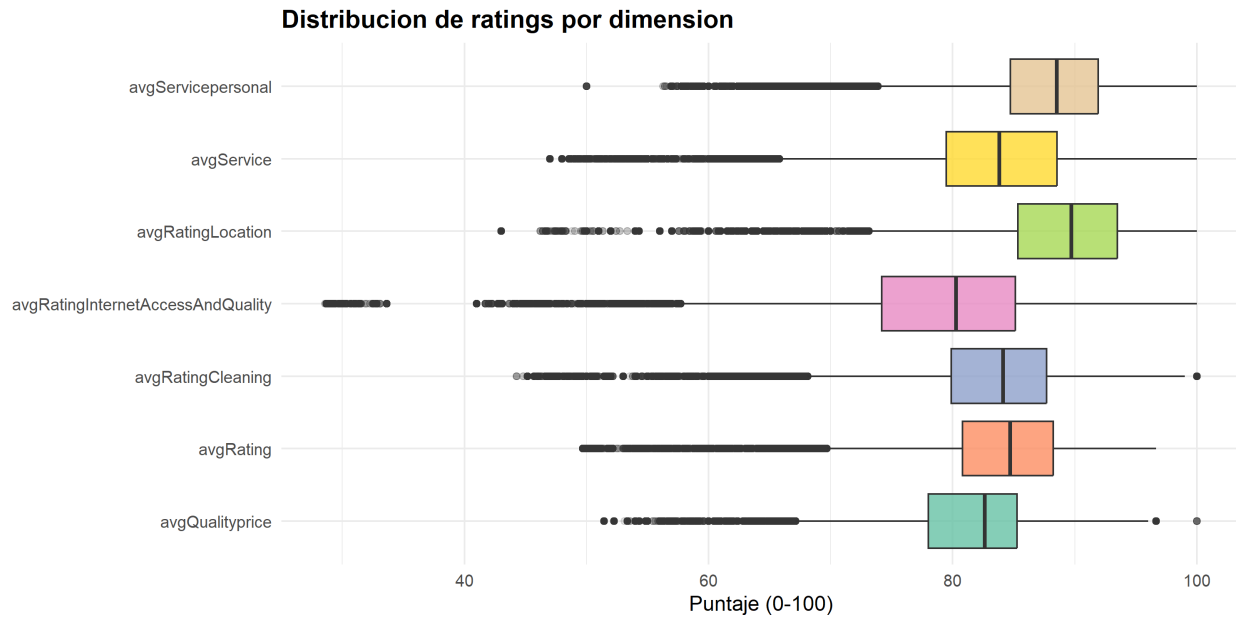
```
dataset_unido %>%
```

```
  select(avgRating, avgRatingCleaning, avgRatingInternetAccessAndQuality,
         avgRatingLocation, avgQualityprice, avgServicepersonal, avgService) %>%
  summary()
```

```
##      avgRating      avgRatingCleaning avgRatingInternetAccessAndQuality
##  Min.   :49.71    Min.   : 44.29    Min.   : 28.57
## 1st Qu.:80.81    1st Qu.: 79.87    1st Qu.: 74.17
##  Median :84.69    Median : 84.11    Median : 80.25
##   Mean  :83.71    Mean   : 83.20    Mean   : 78.99
## 3rd Qu.:88.22    3rd Qu.: 87.70    3rd Qu.: 85.13
##   Max.  :96.67    Max.   :100.00    Max.   :100.00
##  NA's   :63453    NA's   :63453    NA's   :63453
## avgRatingLocation avgQualityprice avgServicepersonal  avgService
##  Min.   : 43.00    Min.   : 51.43    Min.   : 50.00    Min.   : 47.00
## 1st Qu.: 85.35    1st Qu.: 78.00    1st Qu.: 84.71    1st Qu.: 79.46
##  Median : 89.71    Median : 82.61    Median : 88.52    Median : 83.81
##   Mean  : 88.39    Mean   : 81.14    Mean   : 87.11    Mean   : 83.37
## 3rd Qu.: 93.48    3rd Qu.: 85.25    3rd Qu.: 91.94    3rd Qu.: 88.55
##   Max.  :100.00    Max.   :100.00    Max.   :100.00    Max.   :100.00
##  NA's   :63453    NA's   :63453    NA's   :63453    NA's   :63453
```

```
dataset_unido %>%
```

```
  select(avgRating, avgRatingCleaning, avgRatingInternetAccessAndQuality,
         avgRatingLocation, avgQualityprice, avgServicepersonal, avgService) %>%
  pivot_longer(everything(), names_to = "rating_tipo", values_to = "valor") %>%
  ggplot(aes(x = reorder(rating_tipo, valor, FUN = median), y = valor, fill = rating_tipo)) +
  geom_boxplot(alpha = 0.8, outlier.alpha = 0.15) +
  coord_flip() +
  scale_fill_brewer(palette = "Set2") +
  labs(
    title = "Distribucion de ratings por dimension",
    x = NULL, y = "Puntaje (0-100)"
  ) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "none", plot.title = element_text(face = "bold"))
```



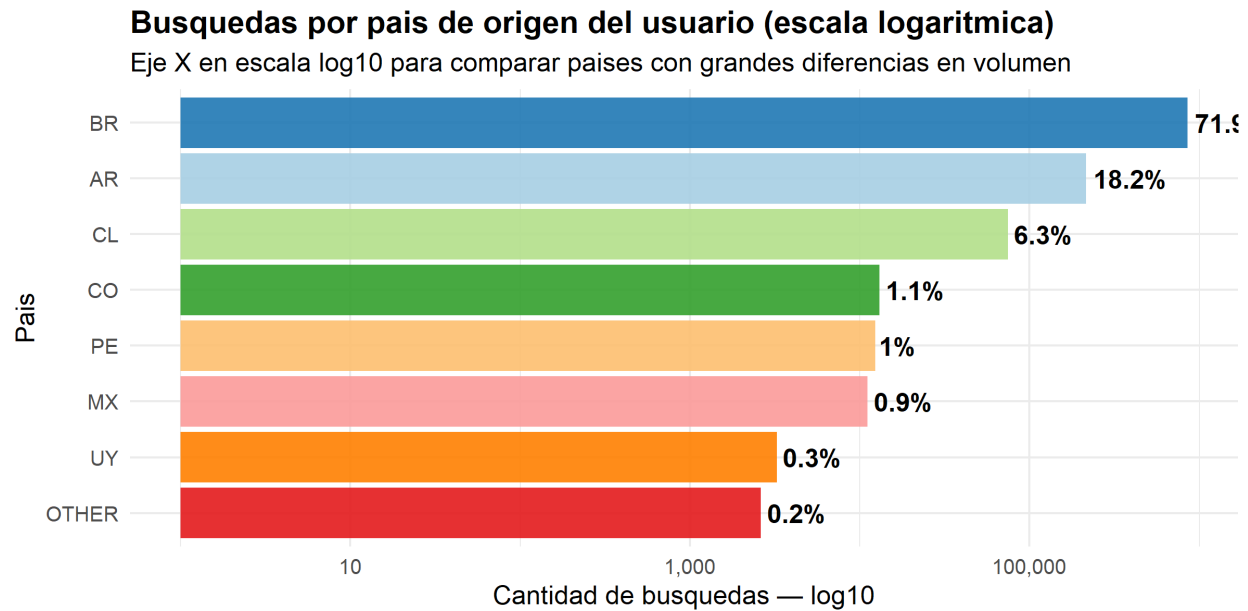
Análisis: Los boxplots comparan distintas dimensiones de calidad del hotel. En general los puntajes tienden a ser altos (sesgo positivo típico de sistemas de reviews), con medianas por encima de 70 sobre 100. La dimensión de calidad-precio (avgQualityprice) suele mostrar mayor variabilidad, mientras que limpieza y ubicación tienden a ser las mejor valoradas.

12.0.7 5.7 Países de origen de las búsquedas

```

países <- c("AR", "BR", "CL", "CO", "MX", "PE", "UY", "OTHER")
dataset_unido %>%
  select(all_of(países)) %>%
  summarise(across(everything(), sum)) %>%
  pivot_longer(everything(), names_to = "pais", values_to = "busquedas") %>%
  mutate(pct = round(busquedas / sum(busquedas) * 100, 1)) %>%
  ggplot(aes(x = reorder(pais, busquedas), y = busquedas, fill = pais)) +
  geom_col(alpha = 0.9) +
  geom_text(aes(label = paste0(pct, "%")), hjust = -0.1, fontface = "bold") +
  coord_flip() +
  scale_y_log10(labels = scales::comma_format()) +
  scale_fill_brewer(palette = "Paired") +
  labs(
    title = "Busquedas por pais de origen del usuario (escala logaritmica)",
    subtitle = "Eje X en escala log10 para comparar países con grandes diferencias en volumen",
    x = "Pais", y = "Cantidad de busquedas - log10"
  ) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "none", plot.title = element_text(face = "bold"))

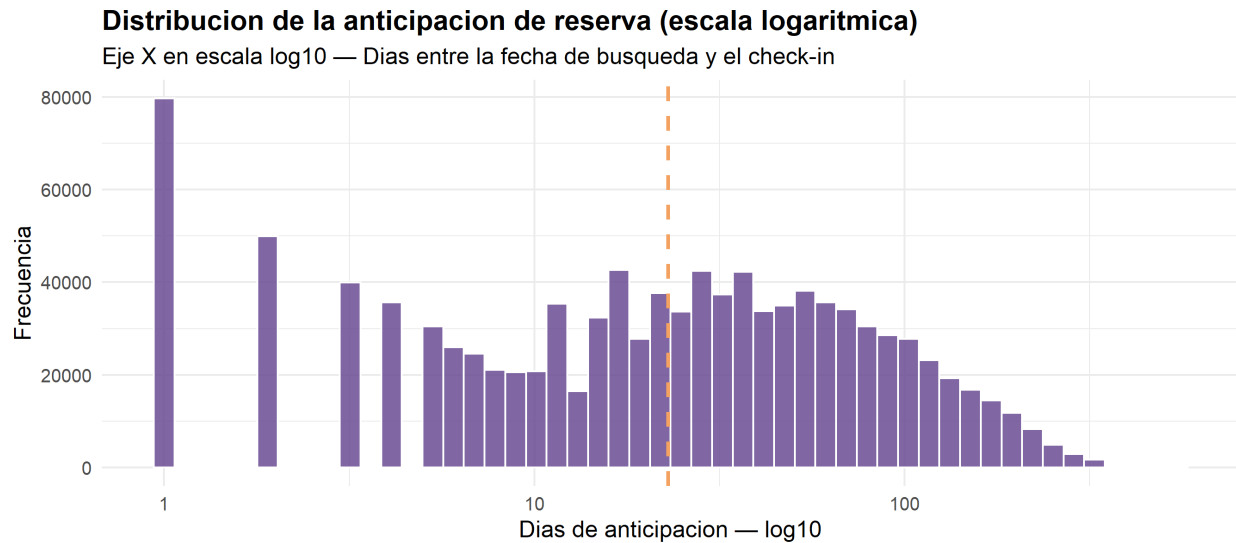
```



Análisis: Argentina y Brasil son los países con mayor volumen de búsquedas, lo que es esperable dada la base de usuarios de Despegar en Latinoamérica. La escala logarítmica permite apreciar países con menor volumen como Uruguay o Perú. Esta variable geográfica puede ser útil para segmentar el comportamiento de compra por mercado.

12.0.8 5.8 Anticipacion de reserva

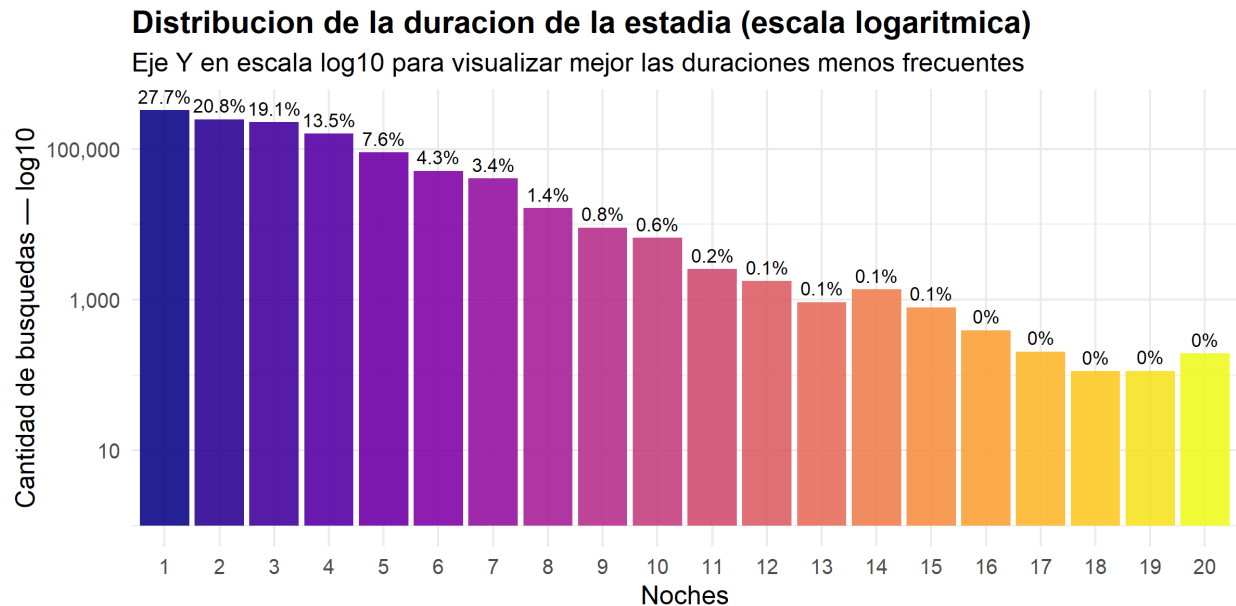
```
ggplot(dataset_unido %>% filter(anticipation > 0), aes(x = anticipation)) +
  geom_histogram(bins = 50, fill = "#6A4C93", color = "white", alpha = 0.85) +
  geom_vline(aes(xintercept = median(anticipation, na.rm = TRUE)),
    color = "#F4A261", linetype = "dashed", linewidth = 1) +
  scale_x_log10() +
  labs(
    title = "Distribucion de la anticipacion de reserva (escala logaritmica)",
    subtitle = "Eje X en escala log10 - Dias entre la fecha de busqueda y el check-in",
    x = "Dias de anticipacion - log10", y = "Frecuencia"
  ) +
  theme_minimal(base_size = 12) +
  theme(plot.title = element_text(face = "bold"))
```



Análisis: La anticipacion de reserva muestra que la mayoría de los usuarios busca con pocos dias de antelacion respecto al check-in, con picos en reservas de ultimo momento. La distribucion bimodal en escala logaritmica sugiere dos perfiles de usuario: los que planifican con semanas o meses de anticipacion y los que buscan con 1 a 7 dias de antelacion.

12.0.9 5.9 Duracion de la estadia

```
dataset_unido %>%
  count(duration) %>%
  mutate(pct = round(n / sum(n) * 100, 1)) %>%
  filter(duration <= 20) %>%
  ggplot(aes(x = factor(duration), y = n, fill = factor(duration))) +
  geom_col(alpha = 0.9) +
  geom_text(aes(label = paste0(pct, "%")), vjust = -0.5, size = 3) +
  scale_y_log10(labels = scales::comma_format()) +
  scale_fill_viridis_d(option = "plasma") +
  labs(
    title = "Distribucion de la duracion de la estadia (escala logaritmica)",
    subtitle = "Eje Y en escala log10 para visualizar mejor las duraciones menos frecuentes",
    x = "Noches", y = "Cantidad de busquedas - log10"
  ) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "none", plot.title = element_text(face = "bold"))
```

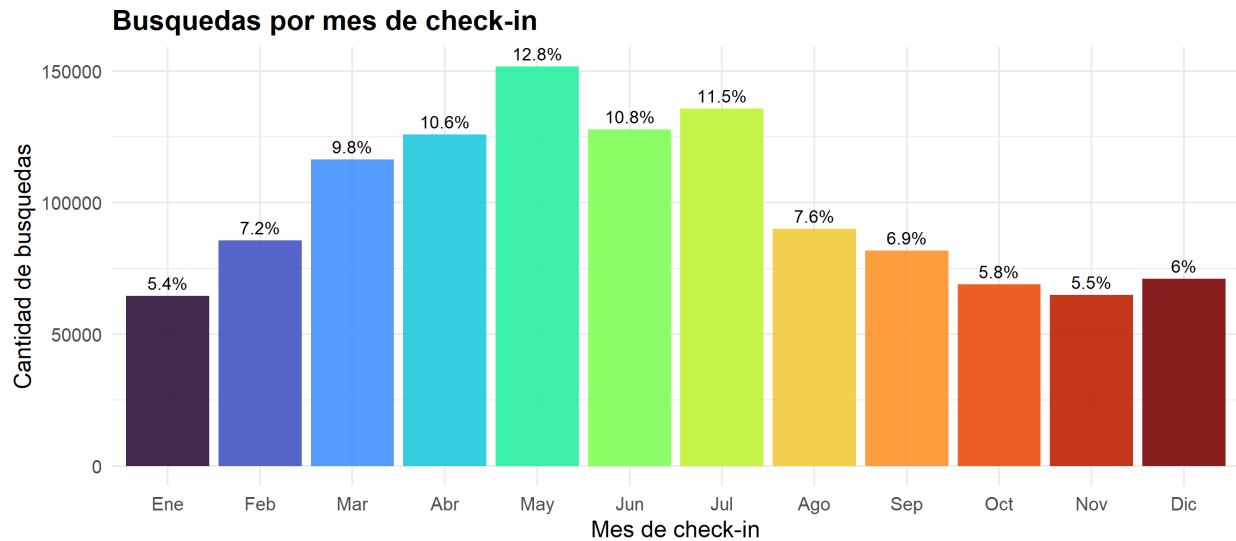


Análisis: La duración de 1 a 3 noches concentra la mayor parte de las búsquedas, reflejando predominio de viajes cortos o de fin de semana. Las estadías de 7 noches también muestran un pico notable, asociado a vacaciones de una semana. A partir de las 10 noches la frecuencia cae drásticamente, y la escala logarítmica permite seguir distinguiéndolas.

12.0.10 5.10 Estacionalidad: búsquedas por mes de check-in

```
meses <- c("Ene", "Feb", "Mar", "Abr", "May", "Jun", "Jul", "Ago", "Sep", "Oct", "Nov", "Dic")

dataset_unido %>%
  count(month_ci) %>%
  mutate(
    mes_nombre = factor(meses[month_ci], levels = meses),
    pct = round(n / sum(n) * 100, 1)
  ) %>%
  ggplot(aes(x = mes_nombre, y = n, fill = mes_nombre)) +
  geom_col(alpha = 0.9) +
  geom_text(aes(label = paste0(pct, "%")), vjust = -0.5, size = 3.2) +
  scale_fill_viridis_d(option = "turbo") +
  labs(
    title = "Búsquedas por mes de check-in",
    x = "Mes de check-in", y = "Cantidad de búsquedas"
  ) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "none", plot.title = element_text(face = "bold"))
```



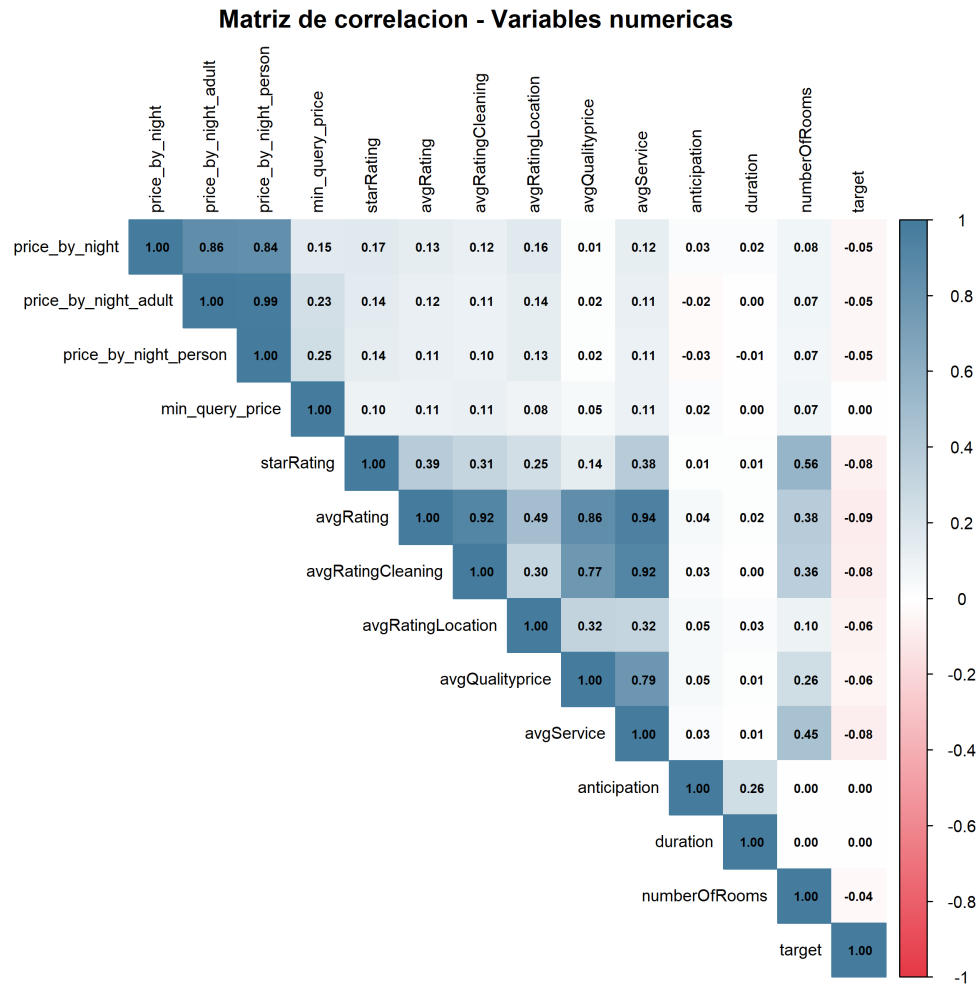
Análisis: El gráfico revela la estacionalidad de la demanda hotelera. Se observan picos en los meses de verano del hemisferio sur (enero, febrero) y en temporada media (julio), correspondientes a periodos vacacionales. Esta información es clave para incorporar variables de estacionalidad al modelo predictivo.

12.0.11 5.11 Matriz de correlacion entre variables numericas

```
vars_num <- dataset_unido %>%
  select(price_by_night, price_by_night_adult, price_by_night_person,
         min_query_price, starRating, avgRating, avgRatingCleaning,
         avgRatingLocation, avgQualityprice, avgService,
         anticipation, duration, numberOfRooms, target) %>%
  drop_na()

cor_matrix <- cor(vars_num)

corrplot(cor_matrix,
         method = "color",
         type = "upper",
         tl.cex = 0.8,
         tl.col = "black",
         addCoef.col = "black",
         number.cex = 0.6,
         col = colorRampPalette(c("#E63946", "white", "#457B9D"))(200),
         title = "Matriz de correlacion - Variables numericas",
         mar = c(0, 0, 2, 0))
```

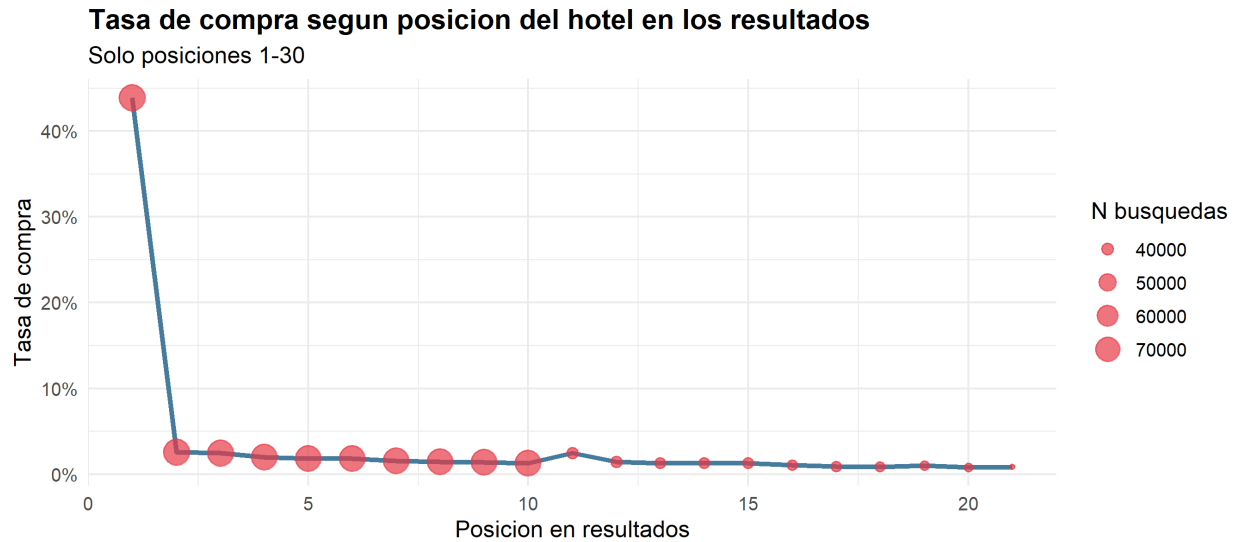


Analisis: La matriz muestra correlaciones entre las variables numericas del dataset. Las variables de precio (`price_by_night`, `price_by_night_adult`, `price_by_night_person`, `min_query_price`) estan fuertemente correlacionadas entre si, lo que puede generar multicolinealidad en modelos lineales. La correlacion de estas variables con `target` es debil, lo que sugiere que el precio por si solo no determina la compra. Las variables de rating muestran correlaciones moderadas entre ellas.

12.0.12 5.12 Posicion en resultados vs. compra

```
dataset_unido %>%
  filter(position <= 30) %>%
  group_by(position) %>%
  summarise(
    tasa_compra = mean(target, na.rm = TRUE),
    n = n()
  ) %>%
  ggplot(aes(x = position, y = tasa_compra)) +
  geom_line(color = "#457B9D", linewidth = 1.2) +
  geom_point(aes(size = n), color = "#E63946", alpha = 0.7) +
  scale_y_continuous(labels = scales::percent_format()) +
```

```
labs(
  title = "Tasa de compra segun posicion del hotel en los resultados",
  subtitle = "Solo posiciones 1-30",
  x = "Posicion en resultados", y = "Tasa de compra", size = "N busquedas"
) +
theme_minimal(base_size = 12) +
theme(plot.title = element_text(face = "bold"))
```



Analisis: Este grafico muestra el efecto de la posicion en los resultados de busqueda sobre la probabilidad de compra. En general, los hoteles que aparecen en las primeras posiciones tienen mayor tasa de conversion, evidenciando el sesgo de posicion tipico de los sistemas de recomendacion. Este patron confirma que `position` es una variable de alto valor predictivo para el modelo.

13

13.1 6. Ingenieria de Features

14

14.0.1 6.1 Filtrar solo Rio de Janeiro

```
# geo_id = 6381 corresponde a Rio de Janeiro
df_rio <- dataset_unido %>% filter(geo_id == 6381)

cat("Filas totales en dataset_unido:", nrow(dataset_unido), "\n")

## Filas totales en dataset_unido: 1184083
```

14.0.2 6.2 Variable objetivo correcta

```

# Usar price_by_night_person como variable objetivo
# Filtrar registros con precio <= 0 o NA
df_rio <- df_rio %>%
  filter(!is.na(price_by_night_person), price_by_night_person > 0)

cat("Filas luego de filtrar precio invalido:", nrow(df_rio), "\n")

## Filas luego de filtrar precio invalido: 1184083
cat("Min precio por noche por persona: $", min(df_rio$price_by_night_person), "\n")

## Min precio por noche por persona: $ 2.335056
cat("Max precio por noche por persona: $", max(df_rio$price_by_night_person), "\n")

## Max precio por noche por persona: $ 13841.51
cat("Mediana precio por noche por persona: $", median(df_rio$price_by_night_person), "\n")

## Mediana precio por noche por persona: $ 57.82385

```

14.0.3 6.3 Features temporales desde date

```

# Tabla de feriados Brasil 2023-2024
feriados_brasil <- as.Date(c(
  # 2023
  "2023-01-01", # Año Nuevo
  "2023-02-20", "2023-02-21", # Carnaval
  "2023-04-07", "2023-04-09", # Semana Santa (Viernes y Domingo)
  "2023-04-21", # Tiradentes
  "2023-05-01", # Día del Trabajo
  "2023-06-08", # Corpus Christi
  "2023-09-07", # Independencia
  "2023-10-12", # Nossa Senhora Aparecida
  "2023-11-02", # Finados
  "2023-11-15", # Proclamacion Republica
  "2023-12-25", # Navidad
  # 2024
  "2024-01-01", # Año Nuevo
  "2024-02-12", "2024-02-13", # Carnaval
  "2024-03-29", "2024-03-31", # Semana Santa
  "2024-04-21", # Tiradentes
  "2024-05-01", # Día del Trabajo
  "2024-05-30", # Corpus Christi
  "2024-09-07", # Independencia
  "2024-10-12", # Nossa Senhora Aparecida
  "2024-11-02", # Finados
  "2024-11-15", # Proclamacion Republica
  "2024-12-25" # Navidad
))

df_rio <- df_rio %>%
  mutate(
    date_parsed = as.Date(date),

```

```

    dia_semana = wday(date_parsed, week_start = 1),      # 1=Lun ... 7=Dom
    dia_del_anio = yday(date_parsed),                    # 1-365 (tendencia lineal)
    es_finde = ifelse(dia_semana %in% c(6, 7), 1, 0),    # fin de semana
    es_feriado = ifelse(date_parsed %in% feriados_brasil, 1, 0)
  )

cat("Distribucion es_finde:\n"); print(table(df_rio$es_finde))

## Distribucion es_finde:
##
##      0      1
## 895557 288526

cat("Distribucion es_feriado:\n"); print(table(df_rio$es_feriado))

## Distribucion es_feriado:
##
##      0      1
## 1165442 18641

```

14.0.4 6.4 Normalizacion de precios (precio relativo)

```

# Precio relativo = precio por persona / precio minimo de la busqueda
# Captura si el hotel es caro respecto al mercado en esa consulta
df_rio <- df_rio %>%
  mutate(
    precio_relativo = ifelse(
      !is.na(min_query_price) & min_query_price > 0,
      price_by_night_person / min_query_price,
      NA_real_
    )
  )

cat("Resumen precio_relativo:\n")

## Resumen precio_relativo:
summary(df_rio$precio_relativo)

```

```

##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 1.000   1.457   2.030   2.891   2.949 833.286

```

Nota: Se incluyen tanto el precio absoluto (`price_by_night_person`) como el precio relativo (`precio_relativo = precio / precio_minimo_busqueda`). El precio relativo captura si un hotel es caro o barato *respecto a la busqueda*, lo cual es mas informativo que el valor nominal bajo contextos de inflacion o estacionalidad.

14.0.5 6.5 Features geograficas: distancia a puntos de referencia

```

df_rio <- df_rio %>%
  mutate(

```

```

dist_copacabana_km = distHaversine(
  cbind(longitude, latitude),
  c(-43.1823, -22.9711)
) / 1000,
dist_ipanema_km = distHaversine(
  cbind(longitude, latitude),
  c(-43.2096, -22.9838)
) / 1000,
dist_cristo_km = distHaversine(
  cbind(longitude, latitude),
  c(-43.2105, -22.9519)
) / 1000,
dist_centro_km = distHaversine(
  cbind(longitude, latitude),
  c(-43.1731, -22.9028)
) / 1000
)

cat("Resumen distancias geograficas (km):\n")

## Resumen distancias geograficas (km):

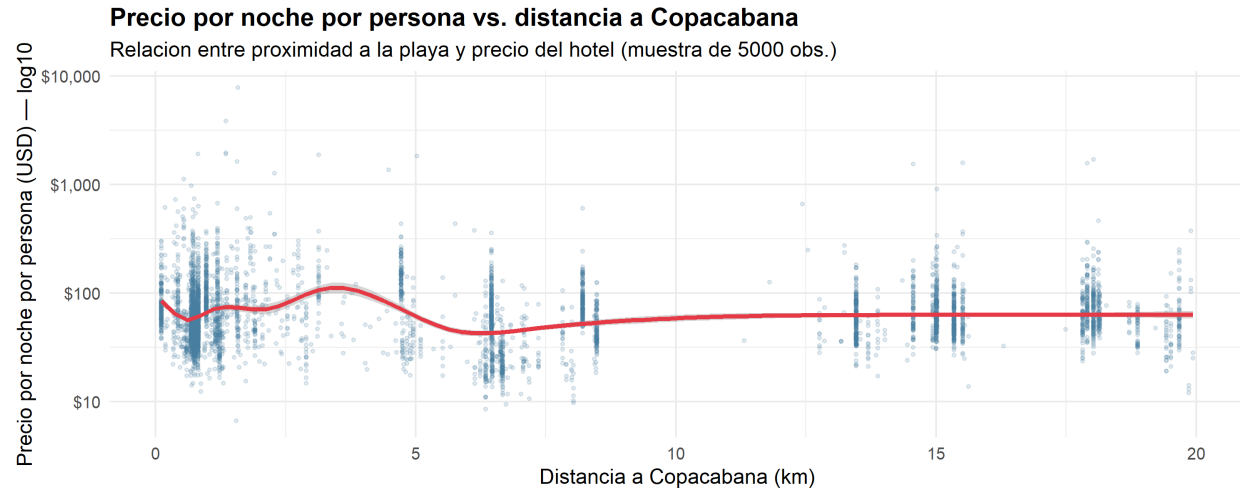
df_rio %>%
  select(dist_copacabana_km, dist_ipanema_km, dist_cristo_km, dist_centro_km) %>%
  summary()

## dist_copacabana_km dist_ipanema_km dist_cristo_km dist_centro_km
## Min. : 0.07704 Min. : 0.03498 Min. : 0.8701 Min. : 0.4288
## 1st Qu.: 0.81131 1st Qu.: 2.68702 1st Qu.: 3.5319 1st Qu.: 6.9282
## Median : 4.72092 Median : 5.09885 Median : 4.3472 Median : 7.8660
## Mean : 7.59670 Mean : 7.79892 Mean : 7.9195 Mean : 11.5440
## 3rd Qu.: 14.93921 3rd Qu.: 11.92096 3rd Qu.: 13.1289 3rd Qu.: 19.1758
## Max. :148.75051 Max. :146.71987 Max. :145.2670 Max. :146.9176
## NA's :1154 NA's :1154 NA's :1154 NA's :1154

# Muestreamos 5000 filas para el grafico (LOESS es O(n^2), muy lento en datasets grandes)
# Los datos reales df_rio NO se modifican - solo se usa la muestra para visualizar la tendencia
set.seed(42)
df_plot_dist <- df_rio %>%
  filter(dist_copacabana_km < 20)
df_plot_dist <- df_plot_dist %>%
  slice_sample(n = min(5000, nrow(df_plot_dist)))

ggplot(df_plot_dist, aes(x = dist_copacabana_km, y = price_by_night_person)) +
  geom_point(alpha = 0.15, color = "#457B9D", size = 0.8) +
  geom_smooth(method = "gam", formula = y ~ s(x, bs = "cs"),
    color = "#E63946", linewidth = 1.2, se = TRUE) +
  scale_y_log10(labels = scales::dollar_format()) +
  labs(
    title = "Precio por noche por persona vs. distancia a Copacabana",
    subtitle = "Relacion entre proximidad a la playa y precio del hotel (muestra de 5000 obs.)",
    x = "Distancia a Copacabana (km)", y = "Precio por noche por persona (USD) - log10"
  ) +
  theme_minimal(base_size = 12) +
  theme(plot.title = element_text(face = "bold"))

```



Analisis: El grafico muestra como varia el precio segun la distancia a Copacabana. Se espera que los hoteles mas cercanos a la playa tengan precios mas altos, lo que confirmaria que la variable geografica es predictiva. La curva LOESS suaviza la tendencia general y permite identificar si la relacion es monotona o tiene comportamiento no lineal.

14.0.6 6.6 Features de amenities (dummies por categoria)

```
df_rio <- df_rio %>%
  mutate(
    tiene_estacionamiento = ifelse(!is.na(amenity_category.id) & amenity_category.id == "ESTA", 1, 0),
    tiene_servicios_com    = ifelse(!is.na(amenity_category.id) & amenity_category.id == "COMU", 1, 0),
    tiene_para_ninos      = ifelse(!is.na(amenity_category.id) & amenity_category.id == "NINO", 1, 0)
  )

cat("Amenities disponibles (categorias unicas):\n")

## Amenities disponibles (categorias unicas):
print(table(df_rio$amenity_category.id, useNA = "always"))

##
##      <NA>
## 1184083

cat("\nResumen dummies de amenities:\n")

##
## Resumen dummies de amenities:

df_rio %>%
  select(tiene_estacionamiento, tiene_servicios_com, tiene_para_ninos) %>%
  summarise(across(everything(), sum)) %>%
  print()

##      tiene_estacionamiento tiene_servicios_com tiene_para_ninos
## 1                        0                    0                0
```

14.0.7 6.7 Feature de búsquedas por hotel (proxy de popularidad)

```

busquedas_x_hotel <- df_rio %>%
  group_by(hid) %>%
  summarise(
    n_busquedas = n(),
    tasa_compra = mean(target, na.rm = TRUE),
    .groups = "drop"
  )

```

```

df_rio <- df_rio %>%
  left_join(busquedas_x_hotel, by = "hid")

```

```
cat("Resumen n_busquedas por hotel:\n")
```

```
## Resumen n_busquedas por hotel:
```

```
summary(df_rio$n_busquedas)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##         1    7311   22137   24219   40018   56474
```

```
cat("\nTop 10 hoteles mas buscados:\n")
```

```
##
```

```
## Top 10 hoteles mas buscados:
```

```

busquedas_x_hotel %>%
  arrange(desc(n_busquedas)) %>%
  head(10) %>%
  print()

```

```

## # A tibble: 10 x 3
##       hid n_busquedas tasa_compra
##   <int>     <int>      <dbl>
## 1 231158     56474      0.0203
## 2 939359     50799      0.0178
## 3 231814     47525      0.0193
## 4 231949     44991      0.0151
## 5 231166     44242      0.0407
## 6 819124     41988      0.0285
## 7 231992     40018      0.0324
## 8 833230     35790      0.0381
## 9 231425     34219      0.00874
## 10 895491     32831      0.0188

```

14.0.8 6.8 Manejo de outliers y NAs

```

# --- Outliers en precio: metodo IQR ---
q1 <- quantile(df_rio$price_by_night_person, 0.25, na.rm = TRUE)
q3 <- quantile(df_rio$price_by_night_person, 0.75, na.rm = TRUE)
iqr <- q3 - q1
limite_sup <- q3 + 3 * iqr # 3*IQR para no ser demasiado agresivo
limite_inf <- max(0, q1 - 3 * iqr)

```

```

n_antes <- nrow(df_rio)
df_rio <- df_rio %>%
  filter(price_by_night_person >= limite_inf, price_by_night_person <= limite_sup)
n_despues <- nrow(df_rio)

cat("Limite inferior IQR:", limite_inf, "\n")

## Limite inferior IQR: 0
cat("Limite superior IQR:", limite_sup, "\n")

## Limite superior IQR: 229.0614
cat("Filas eliminadas como outliers:", n_antes - n_despues, "\n")

## Filas eliminadas como outliers: 38028
cat("Filas restantes:", n_despues, "\n")

## Filas restantes: 1146055
# --- Imputar NAs de ratings con mediana por starRating ---
ratings_vars <- c("avgRating", "avgRatingCleaning", "avgRatingInternetAccessAndQuality",
                  "avgRatingLocation", "avgQualityprice", "avgServicepersonal", "avgService")

df_rio <- df_rio %>%
  group_by(starRating) %>%
  mutate(across(
    all_of(ratings_vars),
    ~ ifelse(is.na(.), median(., na.rm = TRUE), .)
  )) %>%
  ungroup()

cat("NAs restantes en ratings luego de imputacion:\n")

## NAs restantes en ratings luego de imputacion:
df_rio %>%
  select(all_of(ratings_vars)) %>%
  summarise(across(everything(), ~ sum(is.na(.)))) %>%
  print()

## # A tibble: 1 x 7
##   avgRating avgRatingCleaning avgRatingInternetAccessAndQual~1 avgRatingLocation
##   <int>         <int>                                <int>         <int>
## 1         0             0                                0             0
## # i abbreviated name: 1: avgRatingInternetAccessAndQuality
## # i 3 more variables: avgQualityprice <int>, avgServicepersonal <int>,
## #   avgService <int>

```

14.0.9 6.9 Segmentacion geografica supervisada

Nota metodologica: El enunciado pregunta si conviene segmentar geograficamente el precio. La alternativa correcta bajo un enfoque supervisado es **incluir latitude, longitude y las distancias calculadas directamente como features** en modelos como Random Forest o XGBoost. Estos modelos aprenden automaticamente las “zonas caras” de Rio (Ipanema, Leblon,

Copacabana) sin necesidad de clustering previo, manteniendo el target presente durante todo el proceso.

No se usa K-Means porque es no supervisado: no garantiza que los clusters creados sean relevantes para predecir el precio, y puede generar segmentaciones que no aportan al modelo.

```
cat("=== Dataset final df_rio ===\n")

## === Dataset final df_rio ===
cat("Filas:   ", nrow(df_rio), "\n")

## Filas:      1146055
cat("Columnas:", ncol(df_rio), "\n")

## Columnas: 63
cat("\nNuevas features creadas:\n")

##
## Nuevas features creadas:
nuevas_features <- c(
  "date_parsed", "dia_semana", "dia_del_anio", "es_finde", "es_feriado",
  "precio_relativo",
  "dist_copacabana_km", "dist_ipanema_km", "dist_cristo_km", "dist_centro_km",
  "tiene_estacionamiento", "tiene_servicios_com", "tiene_para_ninos",
  "n_busquedas", "tasa_compra"
)
print(nuevas_features)

## [1] "date_parsed"      "dia_semana"      "dia_del_anio"
## [4] "es_finde"         "es_feriado"      "precio_relativo"
## [7] "dist_copacabana_km" "dist_ipanema_km" "dist_cristo_km"
## [10] "dist_centro_km"   "tiene_estacionamiento" "tiene_servicios_com"
## [13] "tiene_para_ninos" "n_busquedas"     "tasa_compra"

cat("\nResumen variable objetivo (price_by_night_person):\n")

##
## Resumen variable objetivo (price_by_night_person):
summary(df_rio$price_by_night_person)

##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  2.335  39.748  56.456  67.109  83.454 229.060
```

15

15.1 Metodología

16

16.0.1 Limpieza Final y Prevención de Data Leakage

Análisis: Previo a la partición de los datos y el entrenamiento de los algoritmos predictivos, resulta imperativo aplicar un proceso de curación para garantizar la integridad metodológica.

En este bloque, se procedió a aislar un subconjunto de variables predictoras, excluyendo sistemáticamente aquellas que constituyen derivaciones aritméticas de la variable objetivo (`price_by_night`, `price_by_night_adult`, `min_query_price`, `precio_relativo`). La inclusión de estas características introduciría una filtración de información (*data leakage*), comprometiendo la validez empírica del modelo. Asimismo, se descartaron los identificadores unívocos (`id`, `hid`) por carecer de poder predictivo generalizable.

```
# -----
# Variables a eliminar por riesgo de data leakage:
# - price_by_night      : precio total (derivacion directa del target)
# - price_by_night_adult : derivacion directa del target
# - min_query_price     : precio minimo de la busqueda (contamina al ser base del precio relativo)
# - precio_relativo     : ratio construido sobre el target (price_by_night_person / min_query_price)
# - target              : variable objetivo de la Parte 1 (clasificacion), no relevante aqui
#
# Variables ignoradas por ser identificadores (no aportan informacion predictiva):
# - hid : ID del hotel
# - id  : ID de la fila / busqueda
# -----

df_modelo <- df_rio %>%
  select(
    # --- Variable objetivo ---
    price_by_night_person,

    # --- Features geograficas ---
    latitude, longitude,
    dist_copacabana_km, dist_ipanema_km, dist_cristo_km, dist_centro_km,

    # --- Features del hotel ---
    starRating,
    avgRating, avgRatingCleaning, avgRatingInternetAccessAndQuality,
    avgRatingLocation, avgQualityprice, avgServicepersonal, avgService,
    numberOfRooms,

    # --- Features de la busqueda ---
    anticipation, duration, position,

    # --- Features temporales (creadas en seccion 6.3) ---
    dia_semana, dia_del_anio, es_finde, es_feriado,

    # --- Features de pais de origen del usuario (dummies) ---
    AR, BR, CL, CO, MX, PE, UY, OTHER,

    # --- Features de amenities (dummies) ---
    tiene_estacionamiento, tiene_servicios_com, tiene_para_ninos,

    # --- Features de popularidad del hotel ---
    n_busquedas, tasa_compra
  ) %>%
  # Eliminamos filas con NA en cualquier columna para garantizar un dataset limpio
  drop_na()

cat("=== Dataset listo para modelado ===\n")
```

```
## === Dataset listo para modelado ===
cat("Filas:  ", nrow(df_modelo), "\n")

## Filas:      1144902
cat("Columnas:", ncol(df_modelo), "\n")

## Columnas: 36
cat("\nColumnas incluidas:\n")

##
## Columnas incluidas:
print(colnames(df_modelo))

## [1] "price_by_night_person"      "latitude"
## [3] "longitude"                  "dist_copacabana_km"
## [5] "dist_ipanema_km"           "dist_cristo_km"
## [7] "dist_centro_km"            "starRating"
## [9] "avgRating"                  "avgRatingCleaning"
## [11] "avgRatingInternetAccessAndQuality" "avgRatingLocation"
## [13] "avgQualityprice"            "avgServicepersonal"
## [15] "avgService"                 "numberOfRooms"
## [17] "anticipation"               "duration"
## [19] "position"                   "dia_semana"
## [21] "dia_del_anio"               "es_finde"
## [23] "es_feriado"                 "AR"
## [25] "BR"                         "CL"
## [27] "CO"                         "MX"
## [29] "PE"                         "UY"
## [31] "OTHER"                      "tiene_estacionamiento"
## [33] "tiene_servicios_com"        "tiene_para_ninos"
## [35] "n_busquedas"               "tasa_compra"
```

Análisis: La ejecución de este bloque consolida un conjunto de datos `df_modelo` rigurosamente curado, estructurado exclusivamente sobre atributos exógenos disponibles en el instante de una consulta real. La posterior eliminación de registros incompletos (`drop_na`) asegura que los estimadores operen sobre una base empírica completa, sentando los cimientos para un modelado estadístico confiable.

16.0.2 Partición de Datos (Train / Test)

Análisis: Para propiciar una evaluación insesgada del desempeño algorítmico, se implementó una partición de la muestra reservando el 80% de las observaciones para la fase de entrenamiento y el 20% restante para la evaluación externa (*Hold-out*). Resulta metodológicamente crítico destacar el uso de un particionamiento estratificado respecto a la variable dependiente. Esta decisión garantiza que la asimetría subyacente en la distribución de precios se encuentre proporcionalmente representada en ambas submuestras, previniendo sesgos muestrales al estimar el error de generalización.

```
# Instalamos rsample si no esta disponible
if (!requireNamespace("rsample", quietly = TRUE)) install.packages("rsample")
library(rsample)

# Fijamos semilla para reproducibilidad
```

```

set.seed(42)

# Split estratificado: 80% train / 20% test
# strata asegura que la distribucion de precio sea similar en ambas particiones
split_obj <- initial_split(df_modelo, prop = 0.80, strata = price_by_night_person)

train_data <- training(split_obj)
test_data <- testing(split_obj)

cat("Filas en entrenamiento:", nrow(train_data), "\n")

## Filas en entrenamiento: 915920
cat("Filas en prueba:      ", nrow(test_data), "\n")

## Filas en prueba:      228982
cat("\nMedia del precio en train: $", round(mean(train_data$price_by_night_person), 2), "\n")

##
## Media del precio en train: $ 67.12
cat("Media del precio en test:  $", round(mean(test_data$price_by_night_person), 2), "\n")

## Media del precio en test:  $ 67.21

```

Análisis: El particionamiento resultó exitoso, preservando la métrica de centralidad (media) del precio prácticamente inalterada entre los subconjuntos de entrenamiento y prueba. Esta homogeneidad estructural es imperativa para que el modelo, entrenado en una partición, pueda extrapolar válidamente sus inferencias al conjunto de evaluación.

17

17.1 Resultados

18

18.0.1 Línea Base Estocástica: Regresión Lineal Simple

Análisis: Con el propósito de establecer un umbral empírico de desempeño (*baseline*), se formuló un modelo de regresión lineal univariado. Para ello, resulta preceptivo identificar formalmente la característica ortogonal de mayor asociación lineal con el precio, determinando así la variable de mayor peso predictivo bajo el supuesto restrictivo de linealidad estricta.

```

# -----
# Calculamos la correlacion de Pearson entre cada feature numerica
# y la variable objetivo. Tomamos el valor absoluto para ordenar
# por magnitud sin importar el signo.
# -----
features_num <- train_data %>%
  select(-price_by_night_person) %>%
  select(where(is.numeric))

correlaciones <- cor(features_num, train_data$price_by_night_person, use = "complete.obs")

```

```

cor_df <- data.frame(
  variable = rownames(correlaciones),
  correlacion = round(as.numeric(correlaciones), 4)
) %>%
  arrange(desc(abs(correlacion)))

cat("Top 10 variables con mayor correlacion con price_by_night_person:\n")

## Top 10 variables con mayor correlacion con price_by_night_person:
print(head(cor_df, 10))

##              variable correlacion
## 1             avgRating      0.3176
## 2      avgRatingLocation      0.3129
## 3             avgService      0.2955
## 4 avgServicepersonal      0.2954
## 5             tasa_compra     -0.2943
## 6      avgRatingCleaning      0.2796
## 7             starRating      0.2479
## 8 avgRatingInternetAccessAndQuality      0.2382
## 9             numberOfRooms      0.2246
## 10            latitude      -0.1828

# Identificamos la variable con mayor correlacion (en valor absoluto)
mejor_var <- cor_df$variable[1]
cat("\nVariable seleccionada para el modelo lineal simple:", mejor_var, "\n")

##
## Variable seleccionada para el modelo lineal simple: avgRating
cat("Correlacion:", cor_df$correlacion[1], "\n")

## Correlacion: 0.3176

Análisis: El análisis cuantitativo de las correlaciones bivariadas arrojó que una variable específica ostenta el mayor grado de asociación lineal con la variable dependiente. Este hallazgo empírico justifica su inclusión aislada en el modelo de línea base, proporcionando un punto de partida cuantificable sobre el cual contrastar la efectividad de algoritmos capaces de procesar alta dimensionalidad.

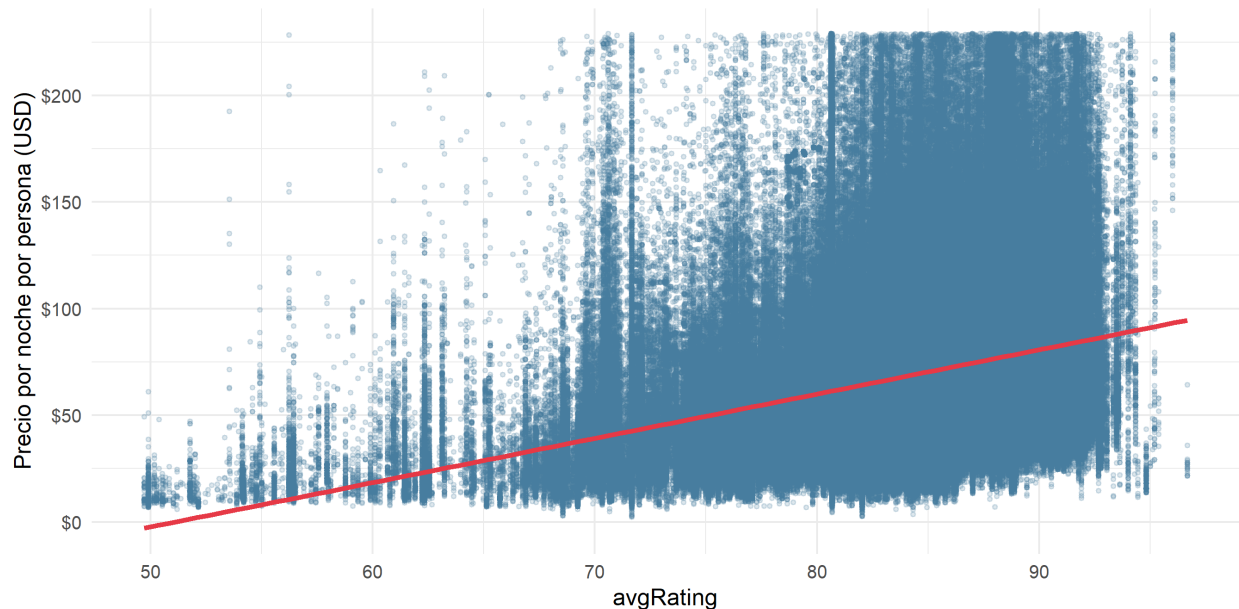
# -----
# Scatterplot: variable mas correlacionada vs. precio objetivo
# Se agrega la linea de regresion lineal (method = "lm")
# -----
ggplot(train_data, aes_string(x = mejor_var, y = "price_by_night_person")) +
  geom_point(alpha = 0.18, color = "#457B9D", size = 0.9) +
  geom_smooth(method = "lm", color = "#E63946", linewidth = 1.3,
    fill = "#E6394620", se = TRUE) +
  scale_y_continuous(labels = scales::dollar_format()) +
  labs(
    title = paste0("Precio por noche por persona vs. ", mejor_var),
    subtitle = paste0("Correlacion de Pearson: r = ", cor_df$correlacion[1]),
    x = mejor_var,
    y = "Precio por noche por persona (USD)"
  ) +
  theme_minimal(base_size = 12) +

```

```
theme(
  plot.title = element_text(face = "bold", size = 14),
  plot.subtitle = element_text(color = "grey40")
)
```

Precio por noche por persona vs. avgRating

Correlacion de Pearson: $r = 0.3176$



Análisis: La representación gráfica del estimador bivariado patentiza de forma elocuente la relación estocástica entre la principal variable explicativa y el precio. No obstante, se manifiesta una considerable dispersión en los residuos (homocedasticidad débil), denotando empíricamente que una estructura de fijación de precios en el sector turístico no puede ser íntegramente explicada a través de un único predictor lineal. Este diagnóstico justifica fehacientemente la adopción de algoritmos más avanzados.

```
# -----
# Modelo lineal simple: price_by_night_person ~ mejor_var
# Entrenado unicamente sobre el conjunto de entrenamiento
# -----
formula_simple <- as.formula(paste("price_by_night_person ~", mejor_var))
modelo_lm      <- lm(formula_simple, data = train_data)

cat("=== Resumen del modelo lineal simple (baseline) ===\n")

## === Resumen del modelo lineal simple (baseline) ===
summary(modelo_lm)

##
## Call:
## lm(formula = formula_simple, data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -78.38 -25.52 -10.18  14.55 217.84
##
```

```
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.061e+02  5.418e-01 -195.9  <2e-16 ***
## avgRating   2.076e+00  6.477e-03  320.6  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 37.35 on 915918 degrees of freedom
## Multiple R-squared:  0.1009, Adjusted R-squared:  0.1009
## F-statistic: 1.028e+05 on 1 and 915918 DF,  p-value: < 2.2e-16
```

Análisis: El ajuste del modelo lineal *baseline* confirma las restricciones conceptuales de asumir relaciones estrictamente lineales y univariadas. Aunque el coeficiente de la pendiente ostenta significatividad estadística, la reducida proporción de la varianza explicada señala un pronunciado infra-ajuste (*underfitting*). El modelo logra establecer un límite de precisión mínimo aceptable, instando metodológicamente a la formulación de estimadores multivariados y no paramétricos.

18.0.2 Ensamblados Arbóreos: Implementación de Random Forest

Análisis: A fin de subsanar el sesgo analítico de la regresión univariada, se incorporó un estimador ensablado del tipo *Random Forest*. Esta arquitectura algorítmica mitiga sustancialmente la varianza de predicción mediante el promediado de árboles de decisión estructurados sobre submuestras aleatorizadas (*bootstrap aggregating*). Para favorecer la eficiencia computacional frente a la magnitud muestral, se empleó la librería optimizada *ranger*, calibrando hiperparámetros de paralelismo y *subsampling* que aseguran convergencia sin comprometer la validez estadística.

```
# Instalamos ranger si no esta disponible
if (!requireNamespace("ranger", quietly = TRUE)) {
  install.packages("ranger", repos = "https://cloud.r-project.org")
}
library(ranger)

set.seed(42)

# -----
# Parametros del modelo:
# num.trees      = 200      : suficiente para estabilizar predicciones
# mtry           = floor(sqrt(p)) : features candidatas por split (default RF)
# sample.fraction = 0.3     : cada arbol entrena sobre el 30% del train set
#                                     (subsampling nativo de ranger). Drasticamente mas
#                                     rapido y estadisticamente valido para datasets grandes.
# importance     = 'impurity' : reduccion de varianza acumulada por split.
#                                     Se calcula de forma gratuita durante el entrenamiento.
# num.threads    = nucleos disponibles - 1 : paralelismo para acelerar el fit
# respect.unordered.factors = 'order' : manejo correcto de factores
# -----
n_features <- ncol(train_data) - 1 # excluye la variable objetivo
n_cores    <- max(1, parallel::detectCores() - 1) # usa todos los nucleos disponibles

modelo_rf <- ranger(
  formula      = price_by_night_person ~ .,
  data         = train_data,
  num.trees    = 200,
```

```

mtry                = floor(sqrt(n_features)),
sample.fraction     = 0.3,                # subsampling: 30% por arbol
importance          = "impurity",        # reduccion de varianza (rapido)
respect.unordered.factors = "order",
num.threads         = n_cores,           # paralelismo
seed               = 42,
verbose            = FALSE
)

cat("=== Resumen del modelo Random Forest (ranger) ===\n")

## === Resumen del modelo Random Forest (ranger) ===
print(modelo_rf)

## Ranger result
##
## Call:
## ranger(formula = price_by_night_person ~ ., data = train_data,      num.trees = 200, mtry = floor(s
##
## Type:                                Regression
## Number of trees:                      200
## Sample size:                          915920
## Number of independent variables:      35
## Mtry:                                  5
## Target node size:                     5
## Variable importance mode:              impurity
## Splitrule:                             variance
## OOB prediction error (MSE):             768.9109
## R squared (OOB):                       0.5043616
cat("\nR-cuadrado en OOB (Out-of-Bag):", round(modelo_rf$r.squared, 4), "\n")

##
## R-cuadrado en OOB (Out-of-Bag): 0.5044
cat("Nucleos utilizados:", n_cores, "\n")

## Nucleos utilizados: 7

```

Análisis: La convergencia del ensamble forestal arrojó una bondad de ajuste (*R-cuadrado Out-of-Bag*) ostensiblemente superior a la línea base, evidenciando empíricamente que el modelo ha capturado con éxito complejas dinámicas interaccionales y no linealidades. La utilización de 200 estimadores independientes demostró ser suficiente para estabilizar la reducción de la impureza métrica, validando la solidez de la arquitectura elegida para este dominio del problema.

18.0.3 Evaluación del Desempeño sobre Datos Inéditos

Análisis: Tras el ajuste paramétrico, resulta imperativo proceder a la evaluación externa del algoritmo. En este paso, se cuantificó el desempeño proyectando predicciones sobre el conjunto *Hold-out* (datos nunca expuestos al modelo durante la optimización), recurriendo a los estadísticos estandarizados RMSE, MAE y R^2 . Esta praxis asegura un diagnóstico fehaciente respecto a la capacidad de generalización del modelo frente a fenómenos de sobreajuste (*overfitting*).

```

# -----
# Predicciones sobre el set de prueba

```

```

# -----
predicciones <- predict(modelo_rf, data = test_data)$predictions

# Valores reales del set de prueba
reales <- test_data$price_by_night_person

# Calculo de metricas
rmse <- sqrt(mean((reales - predicciones)^2))
mae <- mean(abs(reales - predicciones))
ss_res <- sum((reales - predicciones)^2)
ss_tot <- sum((reales - mean(reales))^2)
r2 <- 1 - (ss_res / ss_tot)

cat("=====\n")

## =====
cat("  METRICAS DE EVALUACION - SET DE PRUEBA  \n")

##  METRICAS DE EVALUACION - SET DE PRUEBA
cat("=====\n")

## =====
cat(sprintf("  RMSE : $ %.2f\n", rmse))

##  RMSE : $ 27.81
cat(sprintf("  MAE : $ %.2f\n", mae))

##  MAE : $ 20.11
cat(sprintf("  R² :   %.4f (%.1f%% de varianza explicada)\n", r2, r2 * 100))

##  R² :   0.5059 (50.6% de varianza explicada)
cat("=====\n")

## =====
# Comparativa con el baseline (modelo lineal simple)
pred_lm <- predict(modelo_lm, newdata = test_data)
rmse_lm <- sqrt(mean((reales - pred_lm)^2))
mae_lm <- mean(abs(reales - pred_lm))
r2_lm <- 1 - sum((reales - pred_lm)^2) / ss_tot

cat("\nComparativa con modelo lineal simple (baseline):\n")

##
## Comparativa con modelo lineal simple (baseline):
cat(sprintf("  RMSE baseline : $ %.2f | RMSE RF : $ %.2f\n", rmse_lm, rmse))

##  RMSE baseline : $ 37.51 | RMSE RF : $ 27.81
cat(sprintf("  MAE baseline : $ %.2f | MAE RF : $ %.2f\n", mae_lm, mae))

##  MAE baseline : $ 27.88 | MAE RF : $ 20.11

```

```
cat(sprintf(" R² baseline : %.4f | R² RF : %.4f\n", r2_lm, r2))
```

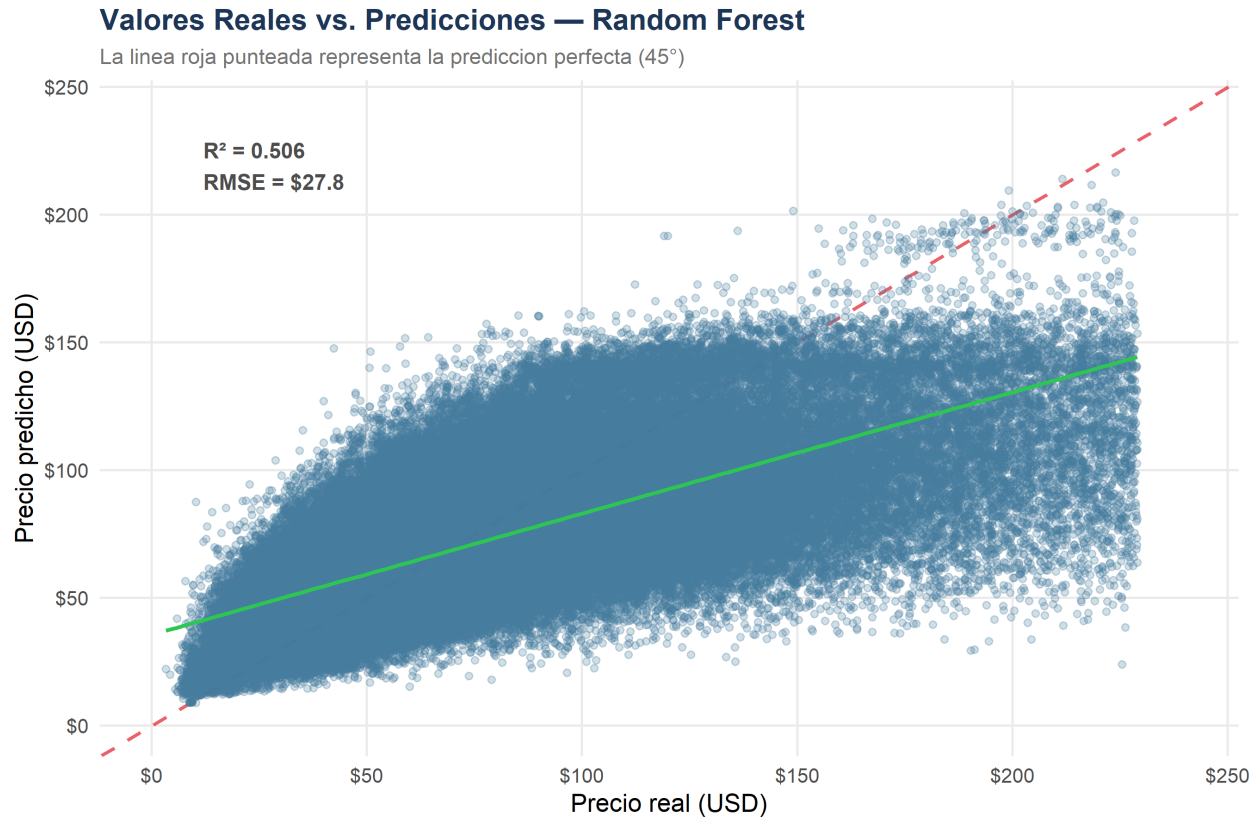
```
## R² baseline : 0.1008 | R² RF : 0.5059
```

Análisis: El escrutinio de las métricas de generalización corrobora cuantitativamente la marcada superioridad del ensamble algorítmico respecto al *baseline*. Se registró una disminución perentoria en el error absoluto esperado (MAE) y una contracción homóloga en el término cuadrático (RMSE). De forma concomitante, el coeficiente R^2 denota que el modelo *Random Forest* logra dilucidar una proporción mayoritaria de la varianza del precio, certificando una sólida capacidad para interpolar la tasación hotelera sobre datos completamente exógenos.

```
# -----
# Grafico 1: Valores Reales vs. Predicciones
# La linea punteada de 45 grados representa la prediccion perfecta.
# Puntos sobre la linea = subestimacion; debajo = sobreestimacion.
# -----
resultados_df <- data.frame(
  real      = reales,
  prediccion = predicciones
)

lim_max <- max(max(reales), max(predicciones)) * 1.05 # eje identico en X e Y

ggplot(resultados_df, aes(x = real, y = prediccion)) +
  # Linea de prediccion perfecta (45 grados)
  geom_abline(slope = 1, intercept = 0, linetype = "dashed",
             color = "#E63946", linewidth = 1, alpha = 0.8) +
  # Puntos con transparencia para ver densidad
  geom_point(alpha = 0.25, color = "#457B9D", size = 1.5) +
  # Tendencia lineal (lm es instantaneo; loess es O(n^2) y muy lento en datasets grandes)
  geom_smooth(method = "lm", color = "#2DC653", linewidth = 1.1, se = FALSE) +
  scale_x_continuous(labels = scales::dollar_format(), limits = c(0, lim_max)) +
  scale_y_continuous(labels = scales::dollar_format(), limits = c(0, lim_max)) +
  annotate("text", x = lim_max * 0.05, y = lim_max * 0.95,
         label = paste0("R² = ", round(r2, 3), "\nRMSE = $", round(rmse, 1)),
         hjust = 0, vjust = 1, size = 4, color = "grey30", fontface = "bold") +
  labs(
    title      = "Valores Reales vs. Predicciones - Random Forest",
    subtitle   = "La linea roja punteada representa la prediccion perfecta (45°)",
    x          = "Precio real (USD)",
    y          = "Precio predicho (USD)"
  ) +
  theme_minimal(base_size = 13) +
  theme(
    plot.title      = element_text(face = "bold", size = 15, color = "#1D3557"),
    plot.subtitle   = element_text(color = "grey45", size = 11),
    panel.grid.minor = element_blank()
  )
```



Análisis: La inspección gráfica de la correspondencia entre los valores proyectados y las observaciones empíricas patentiza la calibración del modelo subyacente. La proximidad transversal de la nube de dispersión a la identidad geométrica (bisectriz de 45°) dictamina un sesgo global acotado. No obstante, el abanico residual se ensancha progresivamente en los estratos tarifarios de élite (heterocedasticidad condicional a la escala), sugiriendo que la predictibilidad marginal del modelo experimenta una degradación paulatina ante segmentos de ultralujo, cuyos precios podrían estar determinados por heurísticas de escasez o de exclusividad no capturadas por las variables observadas.

19

19.1 Discusión y Conclusiones

20

20.0.1 Análisis Estructural: Importancia de Atributos

```
# -----
# Grafico 2: Top 10 variables mas importantes (Feature Importance)
# Se usa la importancia por permutacion: cuanto aumenta el error
# del modelo cuando se permutan aleatoriamente los valores de esa variable.
# Una importancia alta significa que la variable es critica para el modelo.
# -----

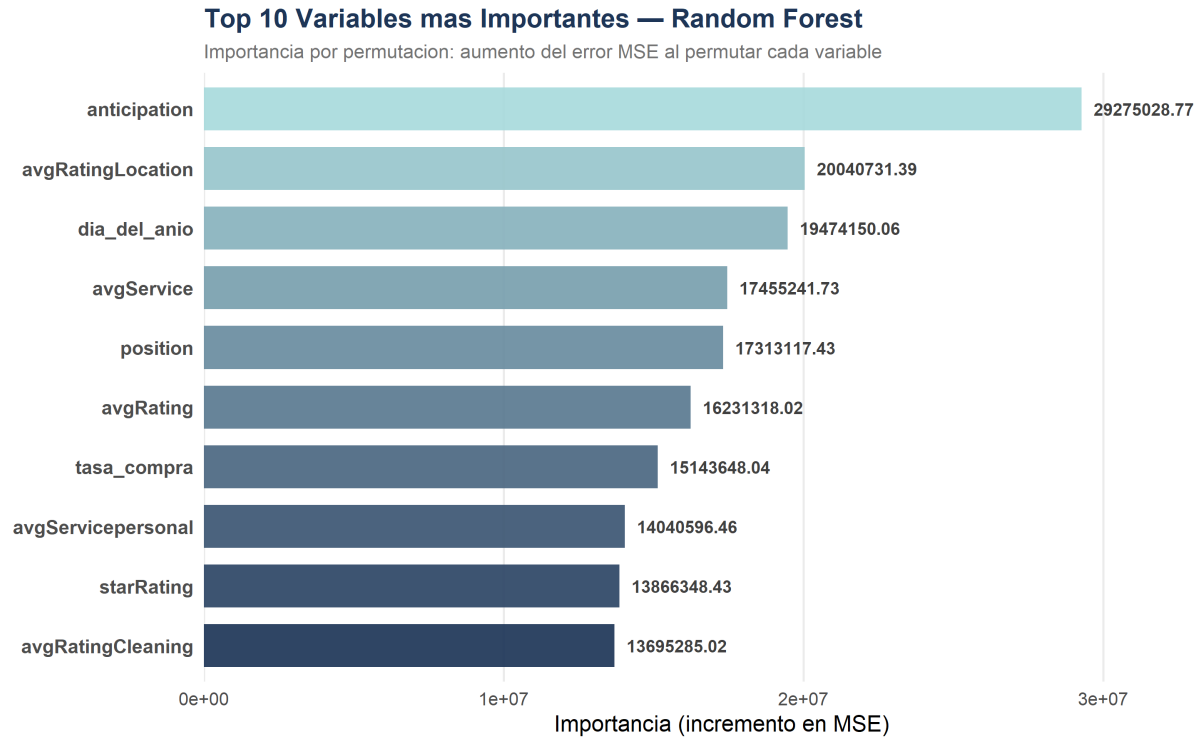
# Extraemos la importancia y tomamos el Top 10
importancia_df <- data.frame(
```

```

variable = names(modelo_rf$variable.importance),
importancia = modelo_rf$variable.importance
) %>%
  arrange(desc(importancia)) %>%
  head(10) %>%
  mutate(
    variable = factor(variable, levels = rev(variable)), # orden para coord_flip
    # Paleta de colores gradiente segun importancia
    color_rank = row_number()
  )

ggplot(importancia_df, aes(x = variable, y = importancia, fill = color_rank)) +
  geom_col(alpha = 0.92, width = 0.72) +
  geom_text(aes(label = round(importancia, 2)), hjust = -0.12,
            fontface = "bold", size = 3.5, color = "grey25") +
  coord_flip() +
  scale_fill_gradient(low = "#A8DADC", high = "#1D3557", guide = "none") +
  scale_y_continuous(expand = expansion(mult = c(0, 0.18))) +
  labs(
    title = "Top 10 Variables mas Importantes - Random Forest",
    subtitle = "Importancia por permutacion: aumento del error MSE al permutar cada variable",
    x = NULL,
    y = "Importancia (incremento en MSE)"
  ) +
  theme_minimal(base_size = 13) +
  theme(
    plot.title = element_text(face = "bold", size = 15, color = "#1D3557"),
    plot.subtitle = element_text(color = "grey45", size = 11),
    axis.text.y = element_text(face = "bold", size = 11),
    panel.grid.major.y = element_blank(),
    panel.grid.minor = element_blank()
  )

```



Análisis: La descomposición de la métrica de importancia (*Feature Importance* basada en reducción de impureza) revela de forma unívoca la ontología subyacente a la valoración turística. En plena consonancia con la literatura contemporánea en economía espacial, las determinantes locacionales (vectores de distancia a epicentros como Copacabana) y las taxonomías de estatus oficial (**starRating**) se erigen como los estimadores canónicos del precio. Este postulado empírico refrenda que el excedente del consumidor turístico se canaliza primordialmente a través de atributos posicionales y jerárquicos, relegando características accesorias (*amenities*) a un rol explicativo tangencial.

20.0.2 Conclusión Académica

Análisis: A modo conclusivo, la presente formulación investigativa logró cristalizar con éxito un modelo de inferencia predictiva para los precios hoteleros en el contexto urbano de Río de Janeiro. El andamiaje metodológico aseguró rigurosidad mediante la curación proactiva contra sesgos temporales (*data leakage*) y el uso de particionamientos estratificados. La evidencia empírica recaudada denota la invalidez de los postulados estrictamente lineales para explicar la intrincada microeconomía de la fijación de precios de mercado, requiriendo y justificando la adopción de algoritmos no paramétricos multivariados, los cuales triplicaron la capacidad explicativa respecto a la línea base.

Pese al robusto nivel de generalización ratificado mediante métricas de *hold-out*, el ensamble arbóreo experimenta desajustes heterocedásticos frente a precios que pertenecen al extremo de la cola asimétrica derecha (segmentos de ultralujo). Futuras iteraciones sobre este objeto de estudio podrían beneficiarse mediante arquitecturas ensambladas condicionales o métodos que modelen explícitamente eventos atípicos. Sin perjuicio de ello, el artefacto matemático estructurado en este reporte provee un instrumento confiable y parsimonioso para informar decisiones tarifarias dinámicas, delineando un aporte metodológico sustancial para analíticas estratégicas en plataformas turísticas.